Yantra Shop — Port Legacy E-Commerce

Knowledge Bytes — Complete Codebase Guide

Generated 2026-10-14 · Vue 3 + TypeScript · .NET 10 Minimal API · EF Core 10 · SQL Server 2022

Section A — Project Orientation

Byte 1: What Yantra Shop is and does

Source: README.md

Focus: Project title and overview

Builds on: None — starting point

In plain terms: Yantra Shop is a small online store: shoppers browse products, fill a cart, check out, and track orders, while admins manage products and orders. It was ported from an older Week-3 MVC 5 application to a modern split stack: a Vue 3 shop front talking to a .NET API.

The code (markdown):

```
# Yantra Shop \u2014 Legacy E-Commerce ported to Vue 3 + .NET 10 Minimal API


Decoupled e-commerce system: a Vue 3 SPA storefront backed by a stateless
ASP.NET Core Minimal API (EF Core 10 + SQL Server), shipped as three Docker
services behind Nginx.
```

What's happening: The README's first lines state the what and the how: a decoupled (frontend and backend run separately) store, a stateless API (no server-side session memory), and three Docker services (db, api, nginx). 'Ported from Week-3 legacy MVC 5' tells you behaviors were preserved, not invented.

Why it matters: This is the mental model for everything after: any feature question is answered by finding its Vue piece, its API piece, and its database piece.

Byte 2: The three-container runtime

Source: docker-compose.yml

Focus: Services overview

Builds on: Byte 1

In plain terms: The whole system runs as three cooperating containers: a database, an API, and a web server. Only the web server faces the outside world; shoppers never talk to the API or database directly.

The code (yaml):

```
services:
  db:     # SQL Server 2022, named volume, no published ports
  api:    # .NET 10 Minimal API, inside the compose network only
  nginx:  # only entrypoint, "${HOST_PORT:-8080}:80" -> port 80
```

What's happening: Compose defines db (data), api (logic), and nginx (front door). Key detail: only nginx publishes a host port; db and api expose ports solely inside the private bridge network, so browsers cannot reach them except through nginx.

Why it matters: If the storefront loads but /api calls fail, this tells you to check nginx proxying and api health first — the browser has no direct API route.

Byte 3: One front door: how Nginx routes each URL

Source: nginx/nginx.conf

Focus: Location blocks and upstream

Builds on: Byte 2

In plain terms: Nginx is a receptionist: static shop pages are served straight from disk, while API, health, and docs URLs are forwarded to the backend container. One special rule makes page refreshes work.

The code (nginx):

```
upstream backend_api {
    server api:8080;
}
location /api/ {
    proxy_pass http://backend_api;
}
location / {
    try_files $uri $uri/ /index.html;
}
```

What's happening: The upstream names the backend (container 'api', port 8080). /api/ traffic is proxied with headers preserved; everything else falls to 'try_files', which serves index.html when no file matches — that is what lets Vue Router history-mode URLs like /products/5 survive a refresh.

Why it matters: Without the fallback, deep links and refreshes on product pages would return 404 from Nginx before Vue ever boots.

Byte 4: Repository map: where everything lives

Source: README.md; backend/Ecommerce.slnx

Focus: Structure block and solution file

Builds on: Byte 1

In plain terms: The repo has four zones: backend (.NET solution), frontend (Vue app), nginx (proxy config), and docs (design notes). Each backend project has one job, listed below.

The code (markdown):

```
backend/Ecommerce.{Api,Core,Infrastructure,Tests}/  .NET solution (Ecommerce.slnx)
frontend/src/{api,components,router,stores,types,utils,views}
nginx/{nginx.conf,Dockerfile}   docker-compose.yml   .env.example   docs/
```

What's happening: Ecommerce.Core holds entities/DTOs/rules; Infrastructure holds the database layer; Api holds endpoints and services; Tests holds xUnit suites. Frontend splits by role (api client, components, views, stores). nginx/ and compose files describe deployment; docs/ records decisions.

Why it matters: This map is the lookup key for the coverage index: any file path in this document resolves to one of these zones.

Byte 5: Boot order: database, then API, then web server

Source: docker-compose.yml

Focus: depends_on and healthchecks

Builds on: Byte 2

In plain terms: Containers start in a strict order enforced by health checks: the database must answer queries before the API starts, and the API must be healthy before Nginx accepts traffic. The API also runs migrations and seeding on every boot.

The code (yaml):

```
api:
  depends_on:
    db:
      condition: service_healthy
nginx:
  depends_on:
    api:
      condition: service_healthy
```

What's happening: depends_on with service_healthy blocks startup until the dependency's healthcheck passes (SQL Server needs ~40 s; the API needs up to 120 s while migrating). The API container then applies EF migrations and idempotent seeding before serving, so a fresh clone self-initializes.

Why it matters: Explains slow first boots and the classic failure: an 'unhealthy api' almost always means the database was not ready or migrations failed — check api logs, not Nginx.

Byte 6: Auth lifecycle at a glance

Source: docs/ARCHITECTURE.md

Focus: Auth sequence diagram

Builds on: Bytes 1, 2

In plain terms: Login returns a short-lived access token plus a long-lived refresh token in a protected cookie. The shop calls APIs with the access token; when it expires, the cookie silently buys a new one — and each refresh token works exactly once.

The code (markdown):

```
SPA->>API: POST /api/auth/login {email,password}
API-->>SPA: 200 {accessToken,user} + HttpOnly refresh_token cookie
SPA->>API: POST /api/auth/refresh (cookie)
API->>DB: revoke old row, insert replacement (rotation)
```

What's happening: Access tokens live only in frontend memory (15 min). Refresh tokens are opaque random strings stored hashed in the database and sent as HttpOnly cookies (JavaScript cannot read them). Refresh rotates: the old row is revoked and a replacement issued, so reusing a token proves theft and yields 401.

Why it matters: This single picture explains login persistence, silent refresh, logout (revoke), and why stealing the cookie buys an attacker at most one use before detection.

Byte 7: Cart-to-order flow at a glance

Source: docs/ARCHITECTURE.md

Focus: Cart merge and checkout transaction

Builds on: Bytes 1, 2

In plain terms: Guests shop with an anonymous cart id; logging in folds the guest cart into the user's cart. Checkout runs inside one database transaction that re-checks prices and stock, writes the order, decrements stock, and empties the cart — or rolls everything back.

The code (markdown):

```
SPA->>API: POST /api/cart/items (X-Cart-Id: guest-uuid)
SPA->>API: POST /api/cart/merge (Bearer + X-Cart-Id)
POST /api/orders (auth): validate -> idempotency check ->
BEGIN TRANSACTION -> ... -> COMMIT
```

What's happening: Prices are never trusted from the client; the server recomputes totals from database prices with legacy shipping tiers plus 8% tax. An idempotency key means retrying checkout returns the original order instead of creating a duplicate.

Why it matters: These two flows are the store's money paths; nearly every edge case in Section H (oversell, double-charge, lost items) is decided inside them.

Byte 8: Secrets live outside git

Source: .env.example

Focus: Environment variable purposes

Builds on: Byte 2

In plain terms: Passwords and signing keys are never committed; they arrive as environment variables documented by .env.example. Two are mandatory and the app refuses to boot without them.

The code (bash):

```
DB_PASSWORD=<secret-value-in-.env-never-committed>
JWT_KEY=<random-32-plus-chars-in-.env>
HOST_PORT=8080
```

What's happening: DB_PASSWORD must satisfy SQL Server complexity (upper/lower/digit/symbol). JWT_KEY signs access tokens and must be 32+ random characters. (Values redacted here by policy; see .env.example for shapes.) Compose uses fail-fast ${VAR:?...} expansion so a missing value aborts startup loudly instead of running insecure. The real .env file is git-ignored; only its key names are documented, never its values.

Why it matters: A 401-on-everything outage after 'docker compose up' almost always traces to a missing or weak JWT_KEY — by design it fails fast.

PUTTING IT TOGETHER — Section A

PUTTING IT TOGETHER — Section A. Yantra Shop is a Vue storefront and a .NET API running as three containers behind one Nginx front door that starts database-first and boots self-seeded. Shoppers get short-lived memory tokens refreshed by one-use cookies, and every cart-to-order step recomputes money from the database inside transactions keyed against double-submit. Secrets arrive from the environment, never from git. With this map in hand, each later section fills in one zone: backend rules, frontend state, deployment plumbing, tests, and the edge cases where the zones meet.

Section B — Backend Foundation

Byte 9: Solution layout and project jobs

Source: backend/Ecommerce.slnx; backend/Ecommerce.Core/Class1.cs; backend/Ecommerce.Infrastructure/Class1.cs

Focus: Solution file and placeholder classes

Builds on: Byte 4

In plain terms: The backend is four projects with one-way dependencies: Core knows nothing, Infrastructure knows Core, Api knows both, Tests knows everything. Two empty Class1.cs files are just template leftovers.

The code (xml):

```
<Solution>
  <Project Path="Ecommerce.Api/Ecommerce.Api.csproj" />
  <Project Path="Ecommerce.Core/Ecommerce.Core.csproj" />
  <Project Path="Ecommerce.Infrastructure/Ecommerce.Infrastructure.csproj" />
  <Project Path="Ecommerce.Tests/Ecommerce.Tests.csproj" />
</Solution>
```

What's happening: The .slnx lists the four projects. Dependency direction matters: entities and interfaces (Core) never reference the database or web layers, so business rules stay testable without a server. Class1.cs files are the default 'dotnet new' placeholders with empty classes and no behavior — safe to ignore.

Why it matters: If you add a reference from Core to Api you break the architecture; the compiler will not stop you, but this rule should.

Byte 10: Api project: framework and packages

Source: backend/Ecommerce.Api/Ecommerce.Api.csproj

Focus: Package references and target framework

Builds on: Byte 9

In plain terms: The API targets .NET 10 and pulls in exactly the libraries its Program.cs features need: JWT auth, validation, SQL Server + SQLite, health checks, logging, and OpenAPI.

The code (xml):

```
<TargetFramework>net10.0</TargetFramework>
<PackageReference Include="FluentValidation" Version="12.0.0" />
<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="10.0.0" />
<PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.0" />
<PackageReference Include="Serilog.AspNetCore" Version="9.0.0" />
```

What's happening: JwtBearer validates access tokens; FluentValidation powers request validators; SqlServer is production storage while Sqlite exists only so tests can run EF in-memory; the OpenApi package generates /openapi/v1.json with no Swashbuckle dependency; Serilog writes structured console logs.

Why it matters: Missing-package confusion is common: if OpenAPI or SQLite behavior looks odd, this list is the ground truth of what is actually referenced.

Byte 11: Core, Infrastructure, and Tests project files

Source: backend/Ecommerce.Core/Ecommerce.Core.csproj; backend/Ecommerce.Infrastructure/Ecommerce.Infrastructure.csproj; backend/Ecommerce.Tests/Ecommerce.Tests.csproj

Focus: Project references and test packages

Builds on: Bytes 9, 10

In plain terms: Core depends on nothing; Infrastructure adds Entity Framework plus password hashing; Tests pulls in xUnit, a test web server, and assertion/mocking libraries.

The code (xml):

```
<!-- Infrastructure -> Core only -->
<ProjectReference Include="..\Ecommerce.Core\Ecommerce.Core.csproj" />
```

```
<PackageReference Include="Microsoft.Extensions.Identity.Core" Version="10.0.0" />
<!-- Tests -->
<PackageReference Include="xunit" Version="2.9.3" />
<PackageReference Include="FluentAssertions" Version="8.0.0" />
<PackageReference Include="Microsoft.AspNetCore.Mvc.Testing" Version="10.0.0" />
<Using Include="Xunit" />
```

What's happening: Infrastructure references Core (entities) and adds Identity.Core solely for PasswordHasher — there is no full ASP.NET Identity system. Tests reference all three projects so WebApplicationFactory can boot the real API; xUnit runs tests, FluentAssertions makes assertions readable, Moq is available for unit doubles.

Why it matters: The one-way reference chain (Tests -> Api -> Infrastructure -> Core) is what keeps domain rules unit-testable in PricingTests without any database.

Byte 12: Category and Product: the catalog entities

Source: backend/Ecommerce.Core/Entities.cs

Focus: Category and Product classes

Builds on: Byte 9

In plain terms: A Category groups products; a Product is the sellable item with price, optional sale price, stock count, and flags controlling visibility. Slugs are the URL-friendly names.

The code (csharp):

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Slug { get; set; } = "";
    public bool IsActive { get; set; } = true;
    public int DisplayOrder { get; set; }
    public ICollection<Product> Products { get; set; } = new List<Product>();
}
public class Product
{
    public int Id { get; set; }
    public string Sku { get; set; } = "";
    public decimal Price { get; set; }
    public decimal? CompareAtPrice { get; set; }
    public int Stock { get; set; }
    public bool IsActive { get; set; } = true;
    public bool IsFeatured { get; set; }
}
```

What's happening: CompareAtPrice is the crossed-out 'was' price; discount % is derived, never stored. IsActive hides items from the storefront without deleting history; IsFeatured drives the 'recommended' sort and home-page rails. Sku/Name/Slug support the case-insensitive search. RowVersion exists but concurrency is handled by transactional checks, not optimistic-concurrency tokens.

Why it matters: Every catalog, cart, and order rule keys off these fields: inactive products 404, zero stock 409s, featured-first ordering.

Byte 13: Customer and Address: who shops and where

Source: backend/Ecommerce.Core/Entities.cs

Focus: Customer and Address classes

Builds on: Byte 12

In plain terms: A Customer is a login identity (email + password hash + role) with saved addresses and orders. An Address is a labeled delivery location, one of which is the default.

The code (csharp):

```
public class Customer
{
    public int Id { get; set; }
    public string Email { get; set; } = "";
    public string PasswordHash { get; set; } = "";
    public string Role { get; set; } = "Customer";
    public ICollection<Address> Addresses { get; set; } = new List<Address>();
}
public class Address
{
    public int CustomerId { get; set; }
    public string FullName { get; set; } = "";
    public string Line1 { get; set; } = "";
    public bool IsDefault { get; set; }
}
```

What's happening: Passwords are never stored — only PasswordHasher output. Role is a plain string ('Customer'/'Admin') checked by the Admin authorization policy, not ASP.NET Identity roles. Email is unique at the database level, which is what makes duplicate registration a 409.

Why it matters: The Customer.Id is the 'sub' claim inside every JWT, so this row is the anchor for carts, orders, addresses, and refresh tokens.

Byte 14: Cart and CartItem: the persisted basket

Source: backend/Ecommerce.Core/Entities.cs

Focus: Cart and CartItem classes

Builds on: Bytes 12, 13

In plain terms: A Cart is a basket owned either by a logged-in customer or by an anonymous guest id — never both. Each CartItem pins the price at add-time so later price changes do not silently rewrite the basket.

The code (csharp):

```
public class Cart
{
    public int? CustomerId { get; set; }
    public string? AnonymousId { get; set; }
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
```

```
    public ICollection<CartItem> Items { get; set; } = new List<CartItem>();
}
public class CartItem
{
    public int ProductId { get; set; }
    public int Quantity { get; set; }
    public decimal UnitPrice { get; set; }
}
```

What's happening: Nullable CustomerId vs AnonymousId is the whole guest story: one cart row per user (unique filtered index) or per guest id. UnitPrice snapshots the price when added, but checkout still re-reads live prices — the snapshot is display data, not the charge basis.

Why it matters: This replaces the legacy session cart: baskets now survive logins, restarts, and server farms because they are database rows, not server memory.

Byte 15: Order snapshots and the idempotency key

Source: backend/Ecommerce.Core/Entities.cs

Focus: Order and OrderItem classes

Builds on: Bytes 12, 14

In plain terms: An Order freezes everything about a purchase — addresses, prices, totals — so later edits to products or addresses cannot rewrite history. The idempotency key makes retried checkouts safe.

The code (csharp):

```
public class Order
{
    public string OrderNumber { get; set; } = "";
    public string Status { get; set; } = "Placed";
    public decimal SubTotal { get; set; }
    public decimal Total { get; set; }
    // Immutable snapshots (preserved even if address records change)
    public string ShipFullName { get; set; } = "";
    public string ShipCity { get; set; } = "";
    public string? IdempotencyKey { get; set; }
    public ICollection<OrderItem> Items { get; set; } = new List<OrderItem>();
}
```

What's happening: Ship*/Bill* columns duplicate the address text at order time (the address book row may later change or be deleted). OrderItems similarly snapshot ProductName/UnitPrice. OrderNumber is a human reference (NE-yyyyMMdd-xxxxxx). IdempotencyKey has a unique filtered index: same key returns the first order instead of inserting a second.

Why it matters: Snapshots are a legal/audit requirement in disguise: without them, editing your address book would falsify old invoices.

Byte 16: RefreshToken: one-use session rows

Source: backend/Ecommerce.Core/Entities.cs

Focus: RefreshToken class and state flags

Builds on: Bytes 6, 13

In plain terms: Each login creates a refresh-token row; each refresh retires the old row and mints a new one. Three computed flags tell whether a presented token is still usable.

The code (csharp):

```
public class RefreshToken
{
    public string TokenHash { get; set; } = "";
    public DateTime ExpiresAt { get; set; }
    public DateTime? RevokedAt { get; set; }
    public string? ReplacedByHash { get; set; }
    public bool IsRevoked => RevokedAt != null;
    public bool IsExpired => DateTime.UtcNow >= ExpiresAt;
    public bool IsActive => !IsRevoked && !IsExpired;
}
```

What's happening: Only the SHA-256 hash is stored, so a database leak does not hand out live sessions. RevokedAt marks rotation/logout; ReplacedByHash chains old to new for reuse detection. IsActive is the single gate checked on every refresh request.

Why it matters: Reuse of a rotated token fails IsActive and yields 401 — that is the theft alarm the whole rotation scheme depends on.

Byte 17: Auth DTOs: the login contract

Source: backend/Ecommerce.Core/Dtos.cs

Focus: Auth records

Builds on: Byte 13

In plain terms: Requests carry only what the endpoint needs (email/password/names); responses return the access token plus the public user profile — never the password hash.

The code (csharp):

```
public record RegisterRequest(string Email, string Password, string FirstName, string LastName);
public record LoginRequest(string Email, string Password);
public record RefreshRequest(string? RefreshToken);
public record AuthResponse(string AccessToken, DateTime ExpiresAt, UserDto User);
```

What's happening: Records give value-type equality and concise syntax for pure data carriers. RefreshRequest.RefreshToken is nullable because callers normally rely on the HttpOnly cookie instead of sending the token in JSON. No entity type ever crosses the API boundary — only these DTOs.

Why it matters: This is why the API can reshape responses without touching the database schema, and vice versa.

Byte 18: Catalog DTOs and the paged envelope

Source: backend/Ecommerce.Core/Dtos.cs

Focus: Category, product, and paging records

Builds on: Byte 17

In plain terms: Product cards carry display fields plus a precomputed discount and stock flag; every product list arrives in a page envelope with counts so the UI can render pagination.

The code (csharp):

```
public record ProductCardDto(int Id, string Name, string Slug, string Description,
    decimal Price, decimal? CompareAtPrice, int? DiscountPct, int Stock, bool InStock,
    string ImageUrl, int CategoryId, string CategoryName);
public record PagedResultDto<T>(IList<T> Items, int Page, int PageSize, int TotalCount, int
TotalPages);
```

What's happening: DiscountPct and InStock are computed server-side so the SPA never does price math. CategoryDto carries ProductCount for nav badges. PagedResultDto<T> is generic — reused for both storefront search and the admin product grid.

Why it matters: The frontend ProductListView/Pagination components bind directly to this envelope's fields.

Byte 19: Cart DTOs: what the basket API returns

Source: backend/Ecommerce.Core/Dtos.cs

Focus: Cart records

Builds on: Bytes 14, 18

In plain terms: Each cart line shows the product, quantity, line total, and how many units remain in stock; the cart footer carries server-computed totals the UI displays but never calculates.

The code (csharp):

```
public record CartItemDto(int ProductId, string ProductName, string Slug, string ImageUrl,
    int Quantity, decimal UnitPrice, decimal LineTotal, int StockAvailable);
public record CartDto(IList<CartItemDto> Items, decimal SubTotal, decimal Shipping,
    decimal Tax, decimal GrandTotal, int ItemCount);
public record AddCartItemRequest(int ProductId, int Quantity);
```

What's happening: StockAvailable lets the quantity inputs cap themselves before the server rejects. Totals (shipping/tax/grand) come from PricingRules on the server; the SPA renders them read-only. Requests contain only ids and quantities — no prices, so clients cannot set their own totals.

Why it matters: Any cart-total bug is a server bug by construction; the client has no vote.

Byte 20: Checkout and order DTOs

Source: backend/Ecommerce.Core/Dtos.cs

Focus: Checkout request and order records

Builds on: Bytes 15, 19

In plain terms: Checkout sends full address snapshots plus method choices and an idempotency key; orders come back with frozen addresses and line items for receipts and history.

The code (csharp):

```
public record CheckoutRequest(AddressInput ShippingAddress, AddressInput BillingAddress,
    int? ShippingAddressId, int? BillingAddressId,
    string ShippingMethod, string PaymentMethod, string? IdempotencyKey);
```

What's happening: AddressInput is the typed address (billing may duplicate shipping when 'same as shipping' is checked). Method strings are validated against PricingRules allow-lists. ShippingAddressId/BillingAddressId optionally link back to the address book without replacing the snapshot.

Why it matters: The frontend CheckoutView must always send an explicit BillingAddress (never null) — a real E2E-caught bug.

Byte 21: Account and admin DTOs

Source: backend/Ecommerce.Core/Dtos.cs

Focus: Profile, address, and admin records

Builds on: Bytes 13, 17

In plain terms: Profile and address DTOs mirror the address book; admin DTOs add merchandising controls (active/featured, slugs, stock) and a dashboard aggregate for the back office.

The code (csharp):

```
public record CreateAddressRequest(string FullName, string Line1, string? Line2,
    string City, string State, string PostalCode, string Country, string? Phone, bool IsDefault);
public record AdminProductRequest(string Name, string Slug, string Description, decimal Price,
    decimal? CompareAtPrice, int Stock, string ImageUrl, int CategoryId, bool IsActive, bool
IsFeatured);
public record DashboardDto(int TotalProducts, int TotalOrders, int TotalCustomers,
    int LowStockProducts, int OutOfStockProducts, int PendingOrders, decimal TotalRevenue);
```

What's happening: Create vs Update address requests differ only in intent; IsDefault drives the exactly-one-default invariant. AdminProductRequest.Slug may be blank (server slugifies the name). DashboardDto is computed live per request — no cached counters to go stale.

Why it matters: Admin screens bind one-to-one to these shapes, so backend validation messages map cleanly onto form fields.

Byte 22: PricingRules: shipping, tax, discount

Source: backend/Ecommerce.Core/Abstractions.cs

Focus: PricingRules shipping and tax methods

Builds on: Byte 12

In plain terms: Shipping is free above per-method thresholds, tax is a flat 8%, and discount percent derives from the two prices. These numbers were copied verbatim from the legacy port.

The code (csharp):

```
public static decimal ShippingFor(decimal subtotal, string? method)
{
    if (subtotal <= 0) return 0;
    var m = (method ?? "standard").ToLowerInvariant();
    if (m == "express") return subtotal >= 150 ? 0 : 12.99m;
    if (m == "priority") return subtotal >= 200 ? 0 : 24.99m;
    return subtotal >= 75 ? 0 : 5.99m;
}
public static decimal TaxFor(decimal subtotal) => Math.Round(subtotal * 0.08m, 2);
```

What's happening: Empty carts ship free (no charge on nothing). Method names are case-insensitive with a standard default. Tax rounds to cents. DiscountPct returns null when there is no genuine markdown (compare-at missing or not higher), so the UI shows no badge.

Why it matters: One static class is the single source of truth used by cart display, checkout charges, and 21 unit tests — change a tier here and both stay consistent.

Byte 23: Sorts, statuses, and order transitions

Source: backend/Ecommerce.Core/Abstractions.cs

Focus: Valid lists and IsValidTransition

Builds on: Byte 22

In plain terms: Search results allow six sort orders; orders move forward through a fixed pipeline with two escape hatches (cancel early, refund late). Backwards moves are rejected.

The code (csharp):

```
public static bool IsValidTransition(string from, string to)
{
    if (from == to) return true;
    var order = new[] { "Placed", "Processing", "Shipped", "Delivered", "Completed" };
    if (to == "Cancelled") return from == "Placed" || from == "Processing";
    if (to == "Refunded") return from == "Delivered" || from == "Completed" || from == "Shipped";
    var fi = Array.IndexOf(order, from); var ti = Array.IndexOf(order, to);
    return fi >= 0 && ti > fi;
}
```

What's happening: Same-status updates are no-ops (allowed). Forward jumps are allowed (Placed straight to Delivered is legal — an integration test asserts this). Cancel is only from Placed/Processing; Refund only after shipping. Unknown statuses fail the ValidStatuses check before this runs.

Why it matters: This tiny state machine is what stops an admin UI slip from resurrecting a delivered order back to Placed.

Byte 24: Service contracts and the DomainException pattern

Source: backend/Ecommerce.Core/Abstractions.cs

Focus: Service interfaces and DomainException

Builds on: Bytes 22, 23

In plain terms: Each feature area promises a small async interface (auth, catalog, cart, orders, account, admin). Business failures throw DomainException carrying the exact HTTP status the endpoint should return.

The code (csharp):

```
public interface ICartService
{
    Task<CartDto> AddItemAsync(int? customerId, string? anonymousId,
        int productId, int quantity, CancellationToken ct = default);
    Task<CartDto> MergeAsync(int customerId, string? anonymousId, CancellationToken ct = default);
}
public class DomainException(string message, int statusCode = 400) : Exception(message)
{
    public int StatusCode { get; } = statusCode;
}
```

What's happening: Guest-or-user duality is visible in every cart signature (customerId plus anonymousId). CancellationTokens flow everywhere for cooperative cancellation. Endpoints catch DomainException and convert it to ProblemDetails with its status — 400 validation, 401 auth, 403 forbidden, 404 missing, 409 conflict — without leaking stack traces.

Why it matters: New business rules should throw DomainException, not return error codes; the HTTP mapping then happens in exactly one place per endpoint.

Byte 25: AppDbContext: the nine tables

Source: backend/Ecommerce.Infrastructure/AppDbContext.cs

Focus: DbSet declarations

Builds on: Bytes 12-16

In plain terms: The context exposes one DbSet per entity — nine tables that mirror the domain model exactly, with no join-table clutter.

The code (csharp):

```
public DbSet<Category> Categories => Set<Category>();
public DbSet<Product> Products => Set<Product>();
public DbSet<Customer> Customers => Set<Customer>();
public DbSet<Address> Addresses => Set<Address>();
public DbSet<Cart> Carts => Set<Cart>();
public DbSet<CartItem> CartItems => Set<CartItem>();
public DbSet<Order> Orders => Set<Order>();
public DbSet<OrderItem> OrderItems => Set<OrderItem>();
public DbSet<RefreshToken> RefreshTokens => Set<RefreshToken>();
```

What's happening: Expression-bodied DbSets are the query entry points services use (db.Products, db.Orders...). The primary-constructor syntax passes options straight to EF. No DbSet for DTOs — DTOs are projections, never tables.

Why it matters: If a table is missing here, no migration will create it; this list and the migration must agree.

Byte 26: EF shape rules: catalog and identity tables

Source: backend/Ecommerce.Infrastructure/AppDbContext.cs

Focus: Category, Product, Customer configuration

Builds on: Byte 25

In plain terms: Slugs and emails are unique, text has max lengths, and money uses exact decimals. Deleting a category with products is blocked at the database level.

The code (csharp):

```
e.HasIndex(x => x.Slug).IsUnique();
e.Property(x => x.Price).HasPrecision(18, 2);
e.HasOne(x => x.Category).WithMany(c => c.Products)
    .HasForeignKey(x => x.CategoryId).OnDelete(DeleteBehavior.Restrict);
e.HasIndex(x => x.Email).IsUnique();
```

What's happening: Unique slug indexes back the admin 'slug already exists' 409s. decimal(18,2) avoids float rounding on money. Restrict (not Cascade) on Product->Category and similar order links means the database refuses deletes that would orphan history — services check first to return friendly 409s instead of raw FK errors.

Why it matters: Schema constraints are the last line of defense when application checks race each other.

Byte 27: EF behavior rules: carts, orders, tokens

Source: backend/Ecommerce.Infrastructure/AppDbContext.cs

Focus: Cart, CartItem, Order, RefreshToken configuration

Builds on: Byte 26

In plain terms: Guest and user carts are each unique-but-optional via filtered indexes; cart contents cascade away with their cart; idempotency keys and order numbers are unique.

The code (csharp):

```
e.HasIndex(x => x.CustomerId).IsUnique().HasFilter("[CustomerId] IS NOT NULL");
e.HasIndex(x => x.AnonymousId).IsUnique().HasFilter("[AnonymousId] IS NOT NULL");
e.HasIndex(x => new { x.CartId, x.ProductId }).IsUnique();
e.HasIndex(x => x.IdempotencyKey).IsUnique().HasFilter("[IdempotencyKey] IS NOT NULL");
```

What's happening: Filtered unique indexes solve the nullable-owner problem: many NULLs allowed, but no two carts for the same user or guest id. The (CartId, ProductId) composite stops duplicate lines per product. Cart->Items cascade deletes lines with the cart; CartItem->Product restricts so history products cannot vanish. TokenHash uniqueness backs rotation integrity.

Why it matters: The merge and idempotent-checkout logic leans on these indexes; without them, duplicate carts and double orders become possible under concurrency.

Byte 28: InitialCreate migration: schema as code

Source: backend/Ecommerce.Infrastructure/Migrations/20261013131852_InitialCreate.cs; backend/Ecommerce.Infrastructure/Migrations/20261013131852_InitialCreate.Designer.cs; backend/Ecommerce.Infrastructure/Migrations/AppDbContextModelSnapshot.cs

Focus: CreateTable calls and model snapshot

Builds on: Bytes 25-27

In plain terms: The migration builds all nine tables with keys, foreign keys, and indexes in dependency order. It is applied automatically at startup — nobody runs SQL by hand.

The code (csharp):

```
migrationBuilder.CreateTable(name: "Categories", ...);
migrationBuilder.CreateTable(name: "Products", ...);   // FK -> Categories
migrationBuilder.CreateTable(name: "Carts", ...);
migrationBuilder.CreateTable(name: "CartItems", ...);  // FK -> Carts, Products
migrationBuilder.CreateTable(name: "Orders", ...);
migrationBuilder.CreateTable(name: "OrderItems", ...); // FK -> Orders, Products
```

What's happening: Categories/Customers come first (referenced tables), then Products/Addresses/Carts/Orders/RefreshTokens, then CartItems/OrderItems. The Designer + ModelSnapshot files are EF-generated bookkeeping describing the same model — read them only to diff what the model looked like at migration time. SeedData calls MigrateAsync before inserting, so Docker boot converges a blank database to current schema plus demo data.

Why it matters: New schema changes mean adding a migration (dotnet ef), never editing this file — it is the historical record of birth of the schema.

Byte 29: SeedData: migrate, then categories

Source: backend/Ecommerce.Infrastructure/SeedData.cs

Focus: EnsureSeededAsync categories block

Builds on: Bytes 25, 28

In plain terms: Seeding is additive and safe to run on every boot: it migrates first, then inserts demo data only when tables are empty — never deleting user data.

The code (csharp):

```
await db.Database.MigrateAsync(ct);
if (!await db.Categories.AnyAsync(ct))
{
    var cats = new[]
    {
        new Category { Name = "Fashion", Slug = "fashion", DisplayOrder = 1 },
        // ... Footwear, Home & Kitchen, Electronics, Beauty, Sports, Books
    };
    db.Categories.AddRange(cats);
    await db.SaveChangesAsync(ct);
```

```
}
```

What's happening: AnyAsync guards make each block idempotent: reboots and restarts skip already-seeded data. Seven categories with stable slugs give the 30 demo products something to attach to. DisplayOrder drives storefront category ordering.

Why it matters: This is why a fresh 'docker compose up' shows a full store with zero manual steps — and why restarts never wipe real orders.

PUTTING IT TOGETHER — Section B

PUTTING IT TOGETHER — Section B (part 1). The domain model names nine tables where catalog, identity, baskets, orders, and sessions live; DTOs draw the API boundary so entities never leak; PricingRules plus the transition table hold the money and lifecycle math every feature reuses; and the DbContext plus its migration turn those decisions into unique indexes, exact decimals, and delete guards. Seed data then gives every fresh boot a working demo store. Startup and validation complete the foundation next: how the process wires, configures, and gates everything built so far.

Section B — Backend Foundation (startup & validation)

Byte 30: SeedData: products and demo logins

Source: backend/Ecommerce.Infrastructure/SeedData.cs

Focus: Products block and demo users

Builds on: Byte 29

In plain terms: Thirty demo products span the seven categories with realistic prices, sale items, and stock levels; two login accounts (admin + customer) are created with hashed passwords.

The code (csharp):

```
("Running Shoes Pro", "running-shoes-pro", "footwear", 2999m, 3999m, 80, true),
("Wireless Headphones", "wireless-headphones", "electronics", 2499m, 3499m, 55, true),
// ... 30 products total
var admin = new Customer { Email = "admin@yantra.local", Role = "Admin" };
admin.PasswordHash = hasher.HashPassword(admin, "Admin!123");
```

What's happening: Products use a tuple table (name, slug, category, price, compare-at, stock, featured) with picsum.photos placeholder images and staggered CreatedAt dates so 'newest' sort looks alive. CompareAtPrice on some items exercises discount badges. The customer also gets one default address in Bengaluru. Passwords go through PasswordHasher — plaintext never touches the database.

Why it matters: Demo credentials in the README (admin@yantra.local / Admin!123) come from here; every E2E test logs in with these accounts.

Byte 31: DesignTimeFactory: migrations without booting the API

Source: backend/Ecommerce.Infrastructure/DesignTimeFactory.cs

Focus: IDesignTimeDbContextFactory implementation

Builds on: Byte 26

In plain terms: Creating a database migration needs a database context but should not start the whole web app. This tiny factory builds one with a fixed connection string purely for design-time tooling.

The code (csharp):

```
public class DesignTimeFactory : IDesignTimeDbContextFactory<AppDbContext>
{
    public AppDbContext CreateDbContext(string[] args)
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseSqlServer("Server=localhost,1433;Database=EcommerceDb;...")
            .Options;
        return new AppDbContext(options);
    }
}
```

What's happening: dotnet ef discovers this class and calls CreateDbContext to scaffold migrations. The connection string here is only used to understand the provider (SQL Server); no connection is opened at design time. Runtime never uses this — Program.cs builds the context from configuration instead.

Why it matters: Without it, every 'dotnet ef migrations add' would try to boot Program.cs (Serilog, JWT, seeding) and fail.

Byte 32: Program: logging and database provider switch

Source: backend/Ecommerce.Api/Program.cs

Focus: Serilog setup and DbContext registration

Builds on: Bytes 10, 26

In plain terms: The app logs structured events to the console and picks SQL Server normally but SQLite when tests say so — one line that makes the entire integration-test strategy possible.

The code (csharp):

```
Log.Logger = new LoggerConfiguration().ReadFrom.Configuration(builder.Configuration)
    .Enrich.FromLogContext().WriteTo.Console().CreateLogger();
if (testSqlite != null)
    builder.Services.AddDbContext<AppDbContext>(o => o.UseSqlite(testSqlite));
else
    builder.Services.AddDbContext<AppDbContext>(o => o.UseSqlServer(conn));
```

What's happening: Serilog replaces default logging with configuration-driven levels plus log-context enrichment. The TestSqlite setting (set only by TestFactory) swaps providers before any service resolves the context, avoiding the dual-provider conflict that broke early tests. The connection string falls back through config, environment, and a dev default.

Why it matters: This switch is why 44 integration tests run in seconds on throwaway in-memory databases instead of needing SQL Server.

Byte 33: Program: wiring services and validators

Source: backend/Ecommerce.Api/Program.cs

Focus: AddScoped registrations and FluentValidation scan

Builds on: Byte 32

In plain terms: Each service interface gets its implementation as a per-request (scoped) lifetime, and all validators are discovered automatically from the assembly.

The code (csharp):

```
builder.Services.AddScoped<ITokenService, TokenService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<ICatalogService, CatalogService>();
builder.Services.AddScoped<ICartService, CartService>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<IAccountService, AccountService>();
builder.Services.AddScoped<IAdminService, AdminService>();
builder.Services.AddValidatorsFromAssemblyContaining<RegisterValidator>();
```

What's happening: Scoped matches the DbContext lifetime (one per request), so services safely share it. AddValidatorsFromAssemblyContaining scans for every AbstractValidator subclass — adding a new validator class with no registration call is enough. Endpoints receive services and validators as parameters via minimal-API dependency injection.

Why it matters: Forgetting to register a new service here produces a runtime DI exception on first call, not a compile error — this list is the checklist.

Byte 34: Program: the CfgOrEnv placeholder trap

Source: backend/Ecommerce.Api/Program.cs

Focus: CfgOrEnv helper

Builds on: Byte 33

In plain terms: Config files contain literal ${PLACEHOLDER} tokens that must be treated as missing so real environment values win. Getting this order wrong caused a real production outage.

The code (csharp):

```
static string CfgOrEnv(IConfiguration cfg, string key, string env, string fallback)
{
    var v = cfg[key];
    // appsettings.json keeps ${PLACEHOLDER} tokens; treat unexpanded tokens as missing.
    if (string.IsNullOrWhiteSpace(v) || v.StartsWith("${"))
        v = Environment.GetEnvironmentVariable(env);
    if (string.IsNullOrWhiteSpace(v) || v.StartsWith("${"))
        v = fallback;
    return v;
}
```

What's happening: Resolution order is config value, then environment variable, then safe fallback — with ${...} tokens explicitly skipped at each step. The original bug returned the dev fallback whenever config was

non-null, so token signing (real env key) and validation (fallback key) diverged and every authenticated call 401'd. The fix made both paths use this helper (TokenService mirrors the same logic).

Why it matters: Any new secret must flow through this helper or its TokenService twin, or Docker deployments will silently disagree with local runs.

Byte 35: Program: JWT validation and the Admin policy

Source: backend/Ecommerce.Api/Program.cs

Focus: AddAuthentication/AddAuthorization

Builds on: Bytes 13, 34

In plain terms: Incoming access tokens must be correctly signed, unexpired, and addressed to this API and its SPA; the Admin policy additionally demands the Admin role for back-office routes.

The code (csharp):

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(o =>
{
    o.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuerSigningKey = true, IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
        ValidateIssuer = true, ValidIssuer = issuer, ValidateAudience = true, ValidAudience = audience,
        ValidateLifetime = true, ClockSkew = TimeSpan.FromMinutes(1),
    };
});
builder.Services.AddAuthorization(opts => opts.AddPolicy("Admin", p => p.RequireRole("Admin")));
```

What's happening: HMAC-SHA256 symmetric signing with a shared secret; issuer 'ecommerce-api' and audience 'ecommerce-spa' must match the claims TokenService writes. One minute of clock skew tolerates container clock drift. The signing key here must equal the one TokenService uses — the Byte-34 incident was exactly this invariant breaking.

Why it matters: A 401 on protected routes with a fresh token means either the key diverged or issuer/audience mismatched; this block is the checklist.

Byte 36: Program: rate limits, CORS, health, OpenAPI

Source: backend/Ecommerce.Api/Program.cs

Focus: AddRateLimiter, AddCors, health checks, OpenAPI

Builds on: Bytes 8, 35

In plain terms: Login endpoints are throttled per IP, browsers are allowed to call from the two known frontends with cookies, and the platform gets health and API-description endpoints for free.

The code (csharp):

```
o.AddPolicy("auth", ctx => RateLimitPartition.GetFixedWindowLimiter(...,
```

```
    _ => new FixedWindowRateLimiterOptions { PermitLimit = authPermit, Window = TimeSpan.FromMinutes(1)
}));
builder.Services.AddCors(o => o.AddPolicy("spa", p => p
    .WithOrigins(... ?? ["http://localhost:5173", "http://localhost:8080"])
    .AllowAnyHeader().AllowAnyMethod().AllowCredentials()));
builder.Services.AddHealthChecks().AddDbContextCheck<AppDbContext>("db");
builder.Services.AddOpenApi();
```

What's happening: Auth allows 20 requests/minute per IP (1000 in Testing so tests never flake). CORS AllowCredentials is mandatory for cookie-based refresh across origins; origins are pinned to dev and Docker URLs. AddDbContextCheck makes /health/db actually probe the database. AddOpenApi emits /openapi/v1.json from route metadata with no extra packages.

Why it matters: A 429 on login means the limiter fired; a browser CORS error means the origin list — both are configured here, not in Nginx.

Byte 37: Program: pipeline, endpoint groups, seed-on-boot

Source: backend/Ecommerce.Api/Program.cs

Focus: Middleware order, Map calls, EnsureSeededAsync

Builds on: Bytes 32-36

In plain terms: Middleware runs in a fixed order (errors, limits, CORS, auth), then six endpoint groups register every route, and finally migrations plus seeding run before the server accepts traffic.

The code (csharp):

```
app.UseRateLimiter();
app.UseCors("spa");
app.UseAuthentication();
app.UseAuthorization();
app.MapGet("/health", () => Results.Ok(new { status = "healthy", time = DateTime.UtcNow
})).AllowAnonymous();
AuthEndpoints.Map(app);
CatalogEndpoints.Map(app);
CartEndpoints.Map(app);
// Apply migrations + seed safely (single-instance; skips when DB unreachable in test envs)
try { await SeedData.EnsureSeededAsync(app.Services); }
catch (Exception ex) { Log.Warning(ex, "Startup seeding skipped: {Msg}", ex.Message); }
```

What's happening: Order matters: CORS before auth, auth before endpoints; UseExceptionHandler (or the dev page in Testing) catches DomainException-unhandled crashes into ProblemDetails. /health is a cheap liveness ping; /health/db gates Nginx. The seed call is wrapped so an unreachable database logs a warning instead of crashing the container — Compose healthchecks then retry the container.

Why it matters: New endpoints must be mapped here or they silently do not exist; the six Map calls are the API's table of contents.

Byte 38: App settings and local launch profiles

Source: backend/Ecommerce.Api/appsettings.json;
backend/Ecommerce.Api/appsettings.Development.json;
backend/Ecommerce.Api/Properties/launchSettings.json

Focus: Jwt lifetimes, origins, Serilog, launch URLs

Builds on: Bytes 34, 35

In plain terms: Both settings files carry placeholder secrets plus 15-minute access / 7-day refresh lifetimes and the two allowed origins; launch profiles give local developers fixed ports outside Docker.

The code (json):

```
{ "Jwt": { "Key": "${JWT_KEY}", "Issuer": "ecommerce-api", "Audience": "ecommerce-spa",
    "AccessTokenMinutes": "15", "RefreshTokenDays": "7" },
  "AllowedOrigins": ["http://localhost:8080", "http://localhost:5173"] }
```

What's happening: appsettings.json and Development are intentionally identical (environment differences come from env vars, not files). Lifetimes are read with ?? fallbacks in TokenService/AuthService, so they are tunable without code changes. launchSettings only affects 'dotnet run' (ports 5263/7227, Development env) — Docker ignores it entirely.

Why it matters: Tuning session length or adding a frontend origin is a config edit here, not a code change — but secrets must still come from the environment per Byte 34.

Byte 39: Validators: registration and login gates

Source: backend/Ecommerce.Api/Validation/Validators.cs

Focus: RegisterValidator and LoginValidator

Builds on: Bytes 17, 33

In plain terms: Registration demands a real email and a strong-ish password (8+ chars with upper, lower, digit); login only checks shape, leaving credential verification to the service.

The code (csharp):

```
RuleFor(x => x.Email).NotEmpty().EmailAddress().MaximumLength(256);
RuleFor(x => x.Password).NotEmpty().MinimumLength(8).MaximumLength(100)
    .Matches(@"[A-Z]").WithMessage("Password must contain an uppercase letter.")
    .Matches(@"[a-z]").WithMessage("Password must contain a lowercase letter.")
    .Matches(@"[0-9]").WithMessage("Password must contain a digit.");
```

What's happening: FluentValidation rules run in endpoints before any database call via ValidateAsync, returning 400 ValidationProblem with per-field messages. Login stays lenient on purpose: 'wrong password' must look identical to 'unknown email' (401 'Invalid email or password') to avoid user enumeration.

Why it matters: Client-side checks mirror these loosely, but these server rules are the enforceable ones — bypassing the UI cannot bypass them.

Byte 40: Validators: cart, checkout, and addresses

Source: backend/Ecommerce.Api/Validation/Validators.cs

Focus: AddCart, UpdateCart, Checkout, AddressInput validators

Builds on: Bytes 19, 20, 39

In plain terms: Quantities are clamped to 0–99, shipping/payment methods must come from the PricingRules allow-lists, and every address needs its six core fields filled.

The code (csharp):

```
RuleFor(x => x.Quantity).InclusiveBetween(1, 99);
RuleFor(x => x.ShippingMethod).NotEmpty()
    .Must(m => new[] { "standard", "express", "priority" }.Contains((m ?? "").ToLower()));
RuleFor(x => x.ShippingAddress).NotNull().SetValidator(new AddressInputValidator());
```

What's happening: Update allows 0 (meaning remove) while Add requires ≥1. Method matching lowercases input, matching the service normalization. The nested AddressInputValidator reuses one address rule set for both shipping and billing — and the same rules separately guard saved-address CRUD via AddressValidator. Services re-validate methods too (defense in depth).

Why it matters: The E2E 'null billing address' bug died here: CheckoutValidator rejects a missing billing address with 400 before any money moves.

Byte 41: Validators: profile and admin inputs

Source: backend/Ecommerce.Api/Validation/Validators.cs

Focus: Profile, Address, AdminProduct, AdminCategory validators

Builds on: Bytes 21, 40

In plain terms: Names are required with sane length caps; admin products need a non-negative price, bounded stock, and a real category; categories need little more than a name.

The code (csharp):

```
RuleFor(x => x.Price).GreaterThanOrEqualTo(0);
RuleFor(x => x.Stock).GreaterThanOrEqualTo(0).LessThanOrEqualTo(100000);
RuleFor(x => x.CategoryId).GreaterThan(0);
RuleFor(x => x.Description).MaximumLength(2000);
```

What's happening: The 100 000 stock cap mirrors the legacy admin limit. Negative prices/stocks are rejected here and again in AdminService (defense in depth, since services are also unit-callable). Slug uniqueness and category existence are service-level checks (they need the database), not validator rules.

Why it matters: Admin UI forms can trust these messages directly — field-level 400s arrive keyed by property name.

PUTTING IT TOGETHER — Section B

PUTTING IT TOGETHER — Section B (part 2). Program.cs assembles logging, provider switching, dependency injection, placeholder-safe secrets, JWT validation, throttling, CORS, health, and docs, then maps six endpoint groups and seeds on boot; FluentValidation guards every write at the door. The backend foundation is now complete: model, rules, schema, data, pipeline, and gates. Section C puts this machinery to work feature by feature, starting with tokens, login flows, catalog reads, and the guest-aware cart.

Section C — Backend Features (auth, catalog, cart)

Byte 42: TokenService: resolving the signing key

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: TokenService Key/Issuer/Audience helpers

Builds on: Bytes 34, 35

In plain terms: The signing key is resolved with the same config-then-environment-then-fallback order as Program.cs, so both sides of JWT (minting and validation) always agree.

The code (csharp):

```
private string Key()
{
    var v = cfg["Jwt:Key"];
    if (string.IsNullOrWhiteSpace(v) || v.StartsWith("${"))
        v = Environment.GetEnvironmentVariable("JWT_KEY");
    if (string.IsNullOrWhiteSpace(v) || v.StartsWith("${"))
        v = "dev-only-signing-key-min-32-chars-0123456789abcdef";
    return v;
}
```

What's happening: The ${-check treats unexpanded appsettings placeholders as absent — the exact fix from the production 401 incident. The dev fallback is deliberately long enough for HS256 (which rejects short keys). Issuer/Audience use the identical pattern via the Value helper.

Why it matters: Two key-resolution sites must stay in sync (here and Program.cs); if tokens suddenly stop validating, diff these first.

Byte 43: CreateAccessToken: claims that identify the shopper

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: TokenService.CreateAccessToken

Builds on: Bytes 13, 42

In plain terms: The access token packs the customer id (twice, for compatibility), email, and role, signed for 15 minutes. It is the shopper's wristband for every protected call.

The code (csharp):

```
var claims = new[]
{
    new Claim(JwtRegisteredClaimNames.Sub, c.Id.ToString()),
    new Claim(ClaimTypes.NameIdentifier, c.Id.ToString()),
    new Claim(ClaimTypes.Role, c.Role),
    new Claim("role", c.Role),
};
var tok = new JwtSecurityToken(Issuer(), Audience(), claims, expires: expires, signingCredentials:
creds);
```

What's happening: Both 'sub' and NameIdentifier carry the id because endpoints read either (CustomerId helper tries both). The role appears as ClaimTypes.Role and 'role' so IsInRole and manual checks agree. Lifetime comes from Jwt:AccessTokenMinutes (default 15). The token is a signed, unencrypted JWT — readable by anyone, forgeable by none without the key.

Why it matters: Never put secrets or the password hash in these claims; they are base64-readable by design.

Byte 44: Refresh tokens: random generation, hashed storage

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: GenerateRefreshToken and HashToken

Builds on: Bytes 16, 42

In plain terms: Refresh tokens are 48 cryptographically random bytes; the database keeps only their SHA-256 hash, so a leak never exposes live sessions.

The code (csharp):

```
public string GenerateRefreshToken() => Convert.ToBase64String(RandomNumberGenerator.GetBytes(48));
public string HashToken(string token)
{
    var bytes = SHA256.HashData(Encoding.UTF8.GetBytes(token));
    return Convert.ToHexString(bytes).ToLowerInvariant();
}
```

What's happening: RandomNumberGenerator is OS-backed crypto randomness (not Random). 48 bytes = 384 bits, far beyond guessing. SHA-256 hashing mirrors password hygiene: verification hashes the presented token and compares hashes. Lowercase hex keeps comparisons culture-proof.

Why it matters: This is why the refresh cookie value is useless to an attacker who only reads the database — and vice versa.

Byte 45: AuthService.Register: new accounts

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: AuthService.RegisterAsync

Builds on: Bytes 13, 17, 42

In plain terms: Registration lowercases the email, rejects duplicates with 409, hashes the password, and immediately logs the user in with fresh tokens.

The code (csharp):

```
var email = req.Email.Trim().ToLowerInvariant();
if (await db.Customers.AnyAsync(c => c.Email == email, ct))
    throw new DomainException("Email is already registered.", 409);
var c = new Customer { Email = email, FirstName = req.FirstName.Trim(), LastName = req.LastName.Trim(),
Role = "Customer" };
c.PasswordHash = _hasher.HashPassword(c, req.Password);
```

What's happening: Lowercasing plus the unique email index makes 'User@X.com' and 'user@x.com' the same account. Role is hardcoded 'Customer' — there is no self-promotion path to Admin. IssueRefreshAsync writes the refresh row and ToAuth mints the access token, so register returns the same AuthResponse shape as login (the frontend treats them identically).

Why it matters: Duplicate-email races beyond the AnyAsync check are still caught by the unique index as a last resort.

Byte 46: AuthService.Login: verify without leaking

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: AuthService.LoginAsync

Builds on: Byte 45

In plain terms: Login looks up the lowercased email and verifies the password hash; unknown email and wrong password produce the identical 401 so attackers cannot probe for accounts.

The code (csharp):

```
var c = await db.Customers.FirstOrDefaultAsync(x => x.Email == email, ct);
if (c == null || _hasher.VerifyHashedPassword(c, c.PasswordHash, req.Password) ==
PasswordVerificationResult.Failed)
    throw new DomainException("Invalid email or password.", 401);
```

What's happening: PasswordHasher.VerifyHashedPassword handles salt and algorithm versioning internally. The single combined condition (null customer OR bad password) guarantees identical timing-shape and message. A successful login issues a new refresh row every time — sessions accumulate until expiry/revocation, enabling multi-device login.

Why it matters: Rate limiting (Byte 36) plus this opaque failure are the brute-force defenses; do not 'improve' the message to say which half failed.

Byte 47: AuthService.Refresh: rotate on every use

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: AuthService.RefreshAsync

Builds on: Bytes 6, 16, 44

In plain terms: Presenting a live refresh token retires it and mints its replacement in one save — the old token can never be used again, which is exactly what detects theft.

The code (csharp):

```
var stored = await db.RefreshTokens.Include(r => r.Customer).FirstOrDefaultAsync(r => r.TokenHash ==
hash, ct);
if (stored == null || !stored.IsActive || stored.Customer == null)
    throw new DomainException("Invalid or expired refresh token.", 401);
// Rotation: revoke old, issue new (detect reuse via revoked lookup)
stored.RevokedAt = DateTime.UtcNow;
```

```
var replacement = new RefreshToken { CustomerId = stored.CustomerId, TokenHash = tokens.HashToken(raw),
ExpiresAt = exp };
stored.ReplacedByHash = replacement.TokenHash;
```

What's happening: Lookup is by hash (Byte 44). Missing, expired, revoked, or ownerless tokens all 401 identically. Old and replacement rows save atomically, chaining ReplacedByHash forReuse forensics. The caller (endpoint) sets the new HttpOnly cookie, completing rotation. An integration test asserts the second use of one token fails.

Why it matters: Reuse-after-rotation failing is a security feature, not a bug — legitimate double-refresh races (two tabs) intentionally force one re-login.

Byte 48: AuthService.Revoke: logout one device or all

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: RevokeAsync and RevokeAllAsync

Builds on: Byte 47

In plain terms: Logout stamps one token revoked; a 'log out everywhere' path revokes all live tokens for a customer. Revoking an already-dead token is a silent no-op.

The code (csharp):

```
public async Task RevokeAsync(string? refreshToken, CancellationToken ct = default)
{
    if (string.IsNullOrWhiteSpace(refreshToken)) return;
    var hash = tokens.HashToken(refreshToken);
    var stored = await db.RefreshTokens.FirstOrDefaultAsync(r => r.TokenHash == hash, ct);
    if (stored != null && !stored.IsRevoked) { stored.RevokedAt = DateTime.UtcNow; await
db.SaveChangesAsync(ct); }
}
```

What's happening: Null/empty input returns without error (logout must be idempotent). RevokeAllAsync loops live tokens for a customer — used implicitly on security-sensitive changes. Access tokens cannot be revoked (they are stateless); they simply expire within 15 minutes, which bounds logout imprecision.

Why it matters: True instant global logout would need a token blocklist; the design accepts a ≤15-minute access tail in exchange for statelessness.

Byte 49: HttpHelpers: identity extraction for endpoints

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: CustomerId, IsAdmin, AnonymousId, ValidateAsync

Builds on: Bytes 24, 43

In plain terms: Endpoints never parse tokens directly; four helpers answer 'who is this?', 'are they admin?', 'which guest cart?', and 'is this request valid?' in one line each.

The code (csharp):

```
public static int? CustomerId(this ClaimsPrincipal u)
```

```
{
    var id = u.FindFirstValue(ClaimTypes.NameIdentifier) ?? u.FindFirstValue("sub");
    return int.TryParse(id, out var n) ? n : null;
}
public static string? AnonymousId(HttpContext ctx)
{
    if (ctx.Request.Headers.TryGetValue("X-Cart-Id", out var v) && !string.IsNullOrWhiteSpace(v))
return v.ToString().Trim()[..Math.Min(64, v.ToString().Trim().Length)];
    if (ctx.Request.Cookies.TryGetValue("cart_id", out var c) && !string.IsNullOrWhiteSpace(c)) return
c.Trim();
    return null;
}
```

What's happening: CustomerId tolerates either claim name and non-numeric garbage (null = anonymous). IsAdmin accepts role claim or 'role' string (both are written at mint). AnonymousId prefers the X-Cart-Id header, falls back to the cart_id cookie, and truncates to the column's 64 chars. ValidateAsync converts FluentValidation failures into 400 ValidationProblem uniformly.

Why it matters: Every cart/order/account endpoint builds on these; a bug here would be a platform-wide auth or cart-identity bug.

Byte 50: AuthEndpoints: register, login, refresh

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: AuthEndpoints.Map register/login/refresh routes

Builds on: Bytes 45-49

In plain terms: Three anonymous routes create sessions: register (201 + Location), login (200), and refresh (cookie or body token, rotated). Auth routes are rate-limited.

The code (csharp):

```
var g = app.MapGroup("/api/auth").WithTags("Auth");
g.MapPost("/register", async (RegisterRequest req, IAuthService auth, IValidator<RegisterRequest> v,
HttpContext ctx) =>
{
    var err = await HttpHelpers.ValidateAsync(v, req);
    if (err != null) return err;
    try
    {
        var (a, refresh, exp) = await auth.RegisterAsync(req);
        SetRefreshCookie(ctx, refresh, exp);
        return Results.Created("/api/auth/me", a);
    }
    catch (DomainException e) { return Results.Problem(e.Message, statusCode: e.StatusCode); }
}).AllowAnonymous().RequireRateLimiting("auth");
```

What's happening: MapGroup prefixes all routes with /api/auth and tags them for OpenAPI. Validation precedes service calls; DomainException maps to ProblemDetails with its status (409/401). Refresh accepts the token from JSON body or cookie, whichever is present. Login mirrors register with 200 instead of 201.

Why it matters: The try/catch-DomainException shape repeats in every endpoint group — it is the codebase's error-handling idiom.

Byte 51: AuthEndpoints: logout, me, and the cookie flags

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: Logout/me routes and SetRefreshCookie

Builds on: Byte 50

In plain terms: Logout revokes the presented token and deletes the cookie; /me rebuilds the public profile from token claims; the cookie is locked down (HttpOnly, Lax, auth-path, HTTPS-aware).

The code (csharp):

```
ctx.Response.Cookies.Append("refresh_token", token, new CookieOptions
{
    HttpOnly = true, Secure = ctx.Request.IsHttps, SameSite = SameSiteMode.Lax, Expires = exp, Path =
"/api/auth",
});
```

What's happening: HttpOnly hides the cookie from JavaScript (XSS cannot steal it). SameSite=Lax allows top-level navigation sends while blocking cross-site POSTs (CSRF mitigation). Path=/api/auth means browsers only send it to auth routes — refresh/logout — shrinking exposure. Secure flips on automatically under HTTPS (production) and stays off for local HTTP dev. /me reads claims only, no database hit.

Why it matters: Cookie theft impact is bounded by HttpOnly + rotation + 7-day expiry; weakening any flag must be a conscious security decision.

Byte 52: CatalogService: categories with counts

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: GetCategoriesAsync

Builds on: Bytes 12, 18

In plain terms: The category nav needs active categories in display order, each with its live product count for badges — two queries fused into one list.

The code (csharp):

```
var cats = await db.Categories.Where(c => c.IsActive).OrderBy(c => c.DisplayOrder).ToListAsync(ct);
var counts = await db.Products.Where(p => p.IsActive).GroupBy(p => p.CategoryId).ToDictionaryAsync(g =>
g.Key, g => g.Count(), ct);
return cats.Select(c => new CategoryDto(c.Id, c.Name, c.Slug, c.Description, c.ImageUrl,
counts.GetValueOrDefault(c.Id))).ToList();
```

What's happening: Inactive categories (and products inside counts) are excluded so deactivated merchandising vanishes instantly. GroupBy-into-dictionary avoids N+1 counting queries. GetValueOrDefault yields 0 for empty categories instead of a missing key crash.

Why it matters: Header nav badges and the home category grid both bind to this; a wrong count here is a visible storefront bug.

Byte 53: CatalogService: search, sort, and paging

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: SearchProductsAsync

Builds on: Bytes 12, 18, 23

In plain terms: Product search filters by category and text, applies one of six sorts, then pages — with guards that normalize every out-of-range input instead of erroring.

The code (csharp):

```
page = Math.Max(1, page);
pageSize = pageSize is < 1 or > 60 ? 12 : pageSize;
if (sort == null || !PricingRules.ValidSorts.Contains(sort)) sort = "recommended";
// ...
q = sort switch
{
    "price_asc" => q.OrderBy(p => p.Price).ThenBy(p => p.Id),
    "price_desc" => q.OrderByDescending(p => p.Price).ThenBy(p => p.Id),
    "newest" => q.OrderByDescending(p => p.CreatedAt).ThenBy(p => p.Id),
    _ => q.OrderByDescending(p => p.IsFeatured).ThenByDescending(p => p.CreatedAt).ThenBy(p => p.Id),
};
```

What's happening: Text search lowercases name/description/SKU for case-insensitive matching (the legacy behavior). Every sort has a deterministic ThenBy(Id) tiebreak so pages never show duplicates or gaps. Page numbers beyond the last page clamp down instead of returning empty — friendlier for stale bookmarks. Only IsActive products are ever queried.

Why it matters: Deterministic ordering is a paging correctness requirement, not a nicety: without the tiebreak, items drift between pages.

Byte 54: CatalogService: product detail and card mapping

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: GetProductAsync and Card mapper

Builds on: Bytes 18, 53

In plain terms: Detail fetch returns one active product with its category name, or null (which becomes 404); the Card mapper centralizes discount, stock-flag, and image-fallback math.

The code (csharp):

```
private static ProductCardDto Card(Product p) => new(
    p.Id, p.Name, p.Slug, p.Description, p.Price, p.CompareAtPrice,
    PricingRules.DiscountPct(p.Price, p.CompareAtPrice), p.Stock, p.Stock > 0,
    string.IsNullOrWhiteSpace(p.ImageUrl) ? "/images/placeholder.png" : p.ImageUrl,
    p.CategoryId, p.Category?.Name ?? "");
```

What's happening: Include(p => p.Category) loads the name in one query. Inactive products return null even if the id exists (soft-hidden, not leaked). The placeholder fallback guarantees the UI always has an image URL. Detail DTO reuses the same shape plus flags — one mapping idiom, two endpoints.

Why it matters: Inactive-product invisibility and the AdminProductsView edit prefill both depend on this exact mapping.

Byte 55: CatalogEndpoints: three public routes

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: CatalogEndpoints.Map

Builds on: Bytes 52-54

In plain terms: The entire public catalog is three anonymous GETs: categories, paged search, and detail-by-id with a 404 for anything missing or hidden.

The code (csharp):

```
var g = app.MapGroup("/api").WithTags("Catalog");
g.MapGet("/categories", async (ICatalogService s) => Results.Ok(await
s.GetCategoriesAsync())).AllowAnonymous();
g.MapGet("/products", async (int? page, int? pageSize, int? categoryId, string? search, string? sort,
ICatalogService s) =>
    Results.Ok(await s.SearchProductsAsync(page ?? 1, pageSize ?? 12, categoryId, search,
sort))).AllowAnonymous();
g.MapGet("/products/{id:int}", async (int id, ICatalogService s) =>
    await s.GetProductAsync(id) is { } p ? Results.Ok(p) : Results.Problem("Product not found.",
statusCode: 404)).AllowAnonymous();
```

What's happening: Defaults (page 1, size 12) apply when query params are absent; the :int route constraint rejects non-numeric ids before the service runs. The 'is { } p' pattern elegantly branches null into 404. No authorization, no validation, no DomainException — catalog reads cannot fail except via infrastructure.

Why it matters: These are the highest-traffic routes; their all-anonymous, cache-friendly GET shape is why Nginx can later cache them safely.

Byte 56: CartService: find-or-create plus totals math

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: GetOrCreateAsync and ToDto

Builds on: Bytes 14, 19, 22, 49

In plain terms: Carts resolve by user id first, guest id second, creating a fresh row when neither exists; totals are recomputed from live product data on every read.

The code (csharp):

```
if (customerId.HasValue) cart = await db.Carts.Include(c => c.Items).ThenInclude(i =>
i.Product).FirstOrDefaultAsync(c => c.CustomerId == customerId, ct);
if (cart == null && !string.IsNullOrWhiteSpace(anon))
    cart = await db.Carts.Include(c => c.Items).ThenInclude(i => i.Product).FirstOrDefaultAsync(c =>
c.AnonymousId == anon, ct);
// ...
var sub = lines.Sum(l => l.LineTotal);
var ship = PricingRules.ShippingFor(sub, "standard");
```

```
var tax = PricingRules.TaxFor(sub);
return new CartDto(lines, sub, ship, tax, sub + ship + tax, lines.Sum(l => l.Quantity));
```

What's happening: Logged-in users always resolve to their user cart (guest id ignored — post-login consistency). New carts store whichever owner is known (user id, else guest id). ToDto includes Product data so names/images render without extra queries; missing products degrade to 'Product' text rather than crashing. Preview shipping always uses 'standard' — checkout recomputes with the chosen method.

Why it matters: Every cart read/write funnels through these two methods, so totals and resolution stay consistent across all six cart operations.

Byte 57: CartService.Add: guarded add-to-basket

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: AddItemAsync

Builds on: Bytes 40, 56

In plain terms: Adding validates quantity bounds, product existence, stock, and merged-quantity overflow — then either bumps an existing line or appends a new one at the live price.

The code (csharp):

```
if (quantity > 99) throw new DomainException("Maximum 99 units per add.", 400);
var product = await db.Products.FirstOrDefaultAsync(p => p.Id == productId && p.IsActive, ct)
    ?? throw new DomainException("Product not found.", 404);
if (product.Stock <= 0) throw new DomainException("This item is out of stock.", 409);
var newQty = quantity + (existing?.Quantity ?? 0);
if (newQty > product.Stock) throw new DomainException($"Only {product.Stock} unit(s) available.", 409);
```

What's happening: Inactive products 404 exactly like missing ones (no information leak about hidden items). Zero-stock is 409, not 400 — a conflict with inventory state, retryable later. The merged-quantity check closes the 'add 50 twice against stock 60' hole. UnitPrice refreshes to the live price on every add so stale snapshots self-heal.

Why it matters: These four guards are the oversell firewall at the cart layer; checkout re-checks because stock moves between add and pay.

Byte 58: CartService: update, remove, clear

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: UpdateItemAsync, RemoveItemAsync, ClearCartAsync

Builds on: Bytes 40, 57

In plain terms: Quantity zero removes the line, quantities re-check stock, and clear wipes all lines — each returning (or resetting) the fresh cart view.

The code (csharp):

```
var item = cart.Items.FirstOrDefault(i => i.ProductId == productId) ?? throw new DomainException("Cart
item not found.", 404);
```

```
if (quantity <= 0) { db.CartItems.Remove(item); await db.SaveChangesAsync(ct); return await
GetCartAsync(customerId, anonymousId, ct); }
if (quantity > 99) throw new DomainException("Maximum 99 units.", 400);
var product = await db.Products.FirstAsync(p => p.Id == productId, ct);
if (quantity > product.Stock) throw new DomainException($"Only {product.Stock} unit(s) available.",
409);
```

What's happening: Update on a missing line 404s (no silent no-op). Remove is idempotent-by-contract at the HTTP layer only for existing lines — missing lines 404 so UIs can distinguish. ClearCartAsync tolerates a nonexistent cart (returns quietly) because post-checkout and logout flows call it unconditionally. UpdatedAt stamps every mutation for future expiry logic.

Why it matters: The quantity-zero-means-remove convention lets the row component use one input for edit and delete.

Byte 59: CartService.Merge: guest basket meets user basket

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs

Focus: MergeAsync

Builds on: Bytes 46, 56

In plain terms: After login, guest lines fold into the user cart with per-line capping at the lower of stock and 99; the guest cart row is then deleted so it can never double-apply.

The code (csharp):

```
foreach (var g in guest.Items)
{
    var product = await db.Products.FirstOrDefaultAsync(p => p.Id == g.ProductId, ct);
    if (product == null || !product.IsActive) continue;
    var ex = mine.Items.FirstOrDefault(i => i.ProductId == g.ProductId);
    var want = g.Quantity + (ex?.Quantity ?? 0);
    var capped = Math.Min(want, Math.Min(product.Stock, 99));
    if (capped <= 0) continue;
    if (ex != null) { ex.Quantity = capped; ex.UnitPrice = product.Price; }
    else mine.Items.Add(new CartItem { CartId = mine.Id, ProductId = g.ProductId, Quantity = capped,
UnitPrice = product.Price });
}
db.Carts.Remove(guest);
```

What's happening: No user cart yet? The guest row is simply adopted (CustomerId set, AnonymousId cleared) — zero item loss. Discontinued/inactive products are skipped, not failed, so one dead item cannot block the merge. Capping favors keeping the sale (partial quantity) over rejecting it. Deleting the guest cart makes merges one-shot; a null/empty guest id short-circuits to the plain user cart.

Why it matters: This is the 'don't lose the shopper's items, don't sell what you don't have' policy — the trickiest three lines in the codebase per the reflection notes.

Byte 60: CartEndpoints: guest-first HTTP surface

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: CartEndpoints.Map and EnsureAnon

Builds on: Bytes 49, 56-59

In plain terms: All cart routes accept guests via X-Cart-Id; mutating routes mint and return a fresh guest id when none was sent, so first-time shoppers work with zero setup.

The code (csharp):

```
private static string? EnsureAnon(HttpContext ctx)
{
    var existing = HttpHelpers.AnonymousId(ctx);
    if (existing != null) return existing;
    var fresh = Guid.NewGuid().ToString("N");
    ctx.Response.Cookies.Append("cart_id", fresh, new CookieOptions { SameSite = SameSiteMode.Lax,
Expires = DateTimeOffset.UtcNow.AddDays(30), Path = "/" });
    ctx.Response.Headers["X-Cart-Id"] = fresh;
    return fresh;
}
```

What's happening: GET /api/cart reads without minting (browsing must not create carts). POST/PUT/DELETE call EnsureAnon, which sets both a 30-day cart_id cookie and an X-Cart-Id response header — the SPA persists the header value to localStorage. POST /cart/merge is the only authorized route (it needs the user id). Quantity validators run before service calls.

Why it matters: The dual cookie+header handoff is why guests keep baskets across reloads even with cookies disabled (header path) or JavaScript storage cleared (cookie path).

PUTTING IT TOGETHER — Section C

PUTTING IT TOGETHER — Section C (part 1). Rotating refresh tokens mint short-lived access; login and registration share one response shape; catalog reads stay public, paged, and deterministic; and carts resolve guest-or-user, guard every quantity against live stock, and merge without losing items or overselling. Orders, accounts, and administration build directly on these primitives next.

Section C — Backend Features (orders, account, administration)

Byte 61: OrderService: validation and idempotency first

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: PlaceOrderAsync validation and duplicate check

Builds on: Bytes 20, 22, 40

In plain terms: Checkout validates methods and addresses, then checks the idempotency key — a retried payment returns the original order instead of charging twice.

The code (csharp):

```
var ship = (req.ShippingMethod ?? "standard").ToLowerInvariant();
```

```
if (!PricingRules.ShippingMethods.Contains(ship)) throw new DomainException("Invalid shipping method.",
400);
ValidateAddress(req.ShippingAddress, "Shipping");
ValidateAddress(req.BillingAddress, "Billing");
var key = string.IsNullOrWhiteSpace(req.IdempotencyKey) ? Guid.NewGuid().ToString("N") :
req.IdempotencyKey.Trim();
var dup = await db.Orders.Include(o => o.Items).FirstOrDefaultAsync(o => o.IdempotencyKey == key, ct);
if (dup != null) return Map(dup);
```

What's happening: Method strings normalize to lowercase before allow-list checks (case-tolerant API). Address validation is amanual second gate behind FluentValidation. A missing key is auto-generated (server-side safety), a supplied key is trimmed, and the duplicate lookup happens before the transaction opens — retries never reach the money code. The same order maps to 201 on retry, indistinguishable from the first placement.

Why it matters: Double-clicks, mobile retries, and flaky networks are the #1 cause of duplicate charges; this check eliminates the entire class.

Byte 62: OrderService: transactional lines at live prices

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: PlaceOrderAsync transaction and line building

Builds on: Byte 61

In plain terms: Inside one database transaction, the cart is reloaded, every line is re-priced from the live product row with a stock check, and totals are computed server-side.

The code (csharp):

```
using var tx = await db.Database.BeginTransactionAsync(ct);
try
{
    var cart = await db.Carts.Include(c => c.Items).FirstOrDefaultAsync(c => c.CustomerId ==
customerId, ct);
    if (cart == null || cart.Items.Count == 0) throw new DomainException("Your cart is empty.", 400);
    decimal subtotal = 0;
    var lines = new List<OrderItem>();
    foreach (var ci in cart.Items)
    {
        var p = await db.Products.FirstOrDefaultAsync(x => x.Id == ci.ProductId, ct);
        if (p == null || !p.IsActive) throw new DomainException("A product in your cart is no longer
available.", 409);
        if (ci.Quantity > p.Stock) throw new DomainException($"Only {p.Stock} unit(s) of '{p.Name}'
available.", 409);
        subtotal += p.Price * ci.Quantity;
        lines.Add(new OrderItem { ProductId = p.Id, ProductName = p.Name, Quantity = ci.Quantity,
UnitPrice = p.Price, LineTotal = p.Price * ci.Quantity });
    }
```

What's happening: Empty carts 400 (checkout of nothing is a client bug). Each line snapshots the live name/price — cart UnitPrice is ignored for charging, closing price-tampering. A product deactivated mid-checkout 409s the whole order rather than silently dropping the line. Everything runs inside an explicit transaction so partial writes are impossible.

Why it matters: Server-side repricing is the anti-fraud core: the client suggests quantities, the server decides money.

Byte 63: OrderService: snapshots, stock, cart clear, commit

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: Order insert, stock decrement, cart clear

Builds on: Byte 62

In plain terms: The order is written with frozen address/product snapshots, stock is decremented, the cart is emptied, and the transaction commits — any failure rolls all of it back.

The code (csharp):

```
var order = new Order
{
    OrderNumber = "NE-" + DateTime.UtcNow.ToString("yyyyMMdd") + "-" + Random.Shared.Next(100000,
999999),
    CustomerId = customerId, OrderDate = DateTime.UtcNow, Status = "Placed",
    SubTotal = subtotal, ShippingCost = shipping, TaxAmount = tax, Total = total,
    ShipFullName = req.ShippingAddress.FullName, ShipLine1 = req.ShippingAddress.Line1,
    BillFullName = req.BillingAddress.FullName, BillLine1 = req.BillingAddress.Line1,
    IdempotencyKey = key,
};
foreach (var l in lines)
{
    var p = await db.Products.FirstAsync(x => x.Id == l.ProductId, ct);
    p.Stock = Math.Max(0, p.Stock - l.Quantity);
}
```

What's happening: OrderNumber uses the new-system 'NE-' prefix (legacy used 'LE-'). Shipping/tax come from PricingRules with the chosen method. Stock clamps at zero (never negative). Two SaveChanges calls (order+stock, then cart clear) share one transaction, committed only at the end; the catch block rolls back and rethrows so endpoints map the original DomainException. Map(order) projects the receipt DTO.

Why it matters: Atomicity here is what 'money moved exactly once' means: no order without stock decrement, no charge without cart clear.

Byte 64: OrderService: history and guarded detail

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: GetHistoryAsync and GetDetailAsync

Builds on: Bytes 21, 63

In plain terms: Order history lists newest-first with item counts; detail shows one order only to its owner (or any admin) — anyone else gets 403.

The code (csharp):

```
public async Task<IList<OrderHistoryDto>> GetHistoryAsync(int customerId, bool isAdmin,
CancellationToken ct = default)
```

```
{
    var q = db.Orders.Include(o => o.Items).AsQueryable();
    if (!isAdmin) q = q.Where(o => o.CustomerId == customerId);
    return await q.OrderByDescending(o => o.OrderDate)
        .Select(o => new OrderHistoryDto(o.Id, o.OrderNumber, o.OrderDate, o.Status, o.Total,
o.Items.Sum(i => i.Quantity)))
        .ToListAsync(ct);
}
```

What's happening: Admins see every order (back-office list); customers see only theirs — enforced in the query, not just the UI. ItemCount sums quantities server-side. GetDetailAsync returns null for missing (endpoint 404s) but throws 403 for cross-customer access, so existence of others' orders is not even confirmable beyond 'forbidden'.

Why it matters: Query-level scoping is the ownership guarantee an integration test explicitly verifies with two users.

Byte 65: OrderEndpoints: three authorized routes

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: OrderEndpoints.Map

Builds on: Bytes 49, 61-64

In plain terms: Checkout plus history plus detail, all requiring login: POST returns 201 with a Location header, GETs return scoped lists and guarded details.

The code (csharp):

```
var g = app.MapGroup("/api/orders").WithTags("Orders").RequireAuthorization();
g.MapPost("/", async (CheckoutRequest req, HttpContext ctx, IOrderService s,
IValidator<CheckoutRequest> v) =>
{
    var err = await HttpHelpers.ValidateAsync(v, req);
    if (err != null) return err;
    var id = ctx.User.CustomerId();
    if (id == null) return Results.Unauthorized();
    try
    {
        var o = await s.PlaceOrderAsync(id.Value, req);
        return Results.Created($"/api/orders/{o.Id}", o);
    }
    catch (DomainException e) { return Results.Problem(e.Message, statusCode: e.StatusCode); }
});
```

What's happening: RequireAuthorization on the group means no guest checkout by design (a documented gap vs legacy). The CustomerId re-check after authorization is belt-and-braces for tokens without id claims. 201 + Location follows REST creation conventions so clients can navigate to the receipt. History/detail endpoints repeat the unauthorized/ownership pattern.

Why it matters: Guest checkout is intentionally absent here — assignment requirement, not oversight; the frontend routes guests to login first.

Byte 66: AccountService: profile and the one-default rule

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: AccountService profile and address methods

Builds on: Bytes 13, 21

In plain terms: Profiles are simple name edits; addresses enforce exactly-one-default: setting a new default demotes the old one, the first address auto-becomes default, and deleting the default promotes a survivor.

The code (csharp):

```
public async Task<AddressDto> CreateAddressAsync(int id, CreateAddressRequest req, CancellationToken ct
= default)
{
    if (req.IsDefault)
        foreach (var a in await db.Addresses.Where(x => x.CustomerId == id &&
x.IsDefault).ToListAsync(ct)) a.IsDefault = false;
    else if (!await db.Addresses.AnyAsync(x => x.CustomerId == id, ct)) req = req with { IsDefault =
true };
```

What's happening: Create uses a 'with' expression to flip IsDefault when the book is empty (records are immutable-ish). Update demotes other defaults only when the edited row becomes default. Delete promotes the first surviving address so the book never ends up default-less. Every query filters by customer id — cross-customer edits return null/false (endpoint 404s) instead of leaking.

Why it matters: The invariant 'every address book has exactly one default' is what lets checkout prefill shipping safely.

Byte 67: AccountEndpoints: profile and address CRUD

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: AccountEndpoints.Map

Builds on: Bytes 49, 66

In plain terms: Six authorized routes cover get/update profile and full address CRUD, with 201+Location on create, 204 on delete, and 404 when an address belongs to someone else.

The code (csharp):

```
g.MapPost("/addresses", async (CreateAddressRequest req, HttpContext ctx, IAccountService s,
IValidator<CreateAddressRequest> v) =>
{
    var err = await HttpHelpers.ValidateAsync(v, req);
    if (err != null) return err;
    var id = ctx.User.CustomerId(); if (id == null) return Results.Unauthorized();
    var a = await s.CreateAddressAsync(id.Value, req);
    return Results.Created($"/api/account/addresses/{a.Id}", a);
});
```

What's happening: The group-level RequireAuthorization plus per-route CustomerId checks mirror the orders pattern. Note the PUT address route skips FluentValidation (update reuses the create shape but

trusts service trimming) — a minor inconsistency worth knowing. Delete returns NoContent on success, Problem-404 otherwise, matching REST delete conventions.

Why it matters: Address ids are unguessable-but-not-secret; ownership scoping (not id secrecy) is the protection, and it lives in the service.

Byte 68: AdminService: live dashboard numbers

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: GetDashboardAsync

Builds on: Bytes 12, 21

In plain terms: The dashboard counts products, orders, customers, low/out-of-stock items, pending orders, and non-cancelled revenue with direct queries — no caches, always current.

The code (csharp):

```
var low = await db.Products.CountAsync(p => p.Stock > 0 && p.Stock < 10, ct);
var oos = await db.Products.CountAsync(p => p.Stock <= 0, ct);
var pending = await db.Orders.CountAsync(o => o.Status == "Placed", ct);
var revenue = await db.Orders.Where(o => o.Status != "Cancelled" && o.Status != "Refunded").SumAsync(o
=> (decimal?)o.Total, ct) ?? 0;
```

What's happening: Low-stock threshold (<10, still positive) separates 'reorder soon' from 'gone'. Revenue excludes cancelled/refunded orders and null-coalesces to 0 on an empty store. Each metric is one COUNT/SUM query — seven round trips, trivially fast at this scale and trivially correct (no aggregation jobs to break).

Why it matters: Per-day sales series and best-sellers from legacy were deliberately trimmed; what remains is guaranteed accurate because it is computed, not stored.

Byte 69: AdminService: product CRUD with slug discipline

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: Create/Update/DeleteProductAsync and Slugify

Builds on: Bytes 18, 27, 40

In plain terms: Admins create and edit products with auto-generated URL slugs; deletes are refused when order history or live carts reference the product — deactivate instead.

The code (csharp):

```
private static string Slugify(string name) =>
    string.Join("-", System.Text.RegularExpressions.Regex.Split(name.ToLowerInvariant(), @"[^a-z0-
9]+").Where(s => s.Length > 0));
if (await db.Products.AnyAsync(p => p.Slug == slug, ct)) throw new DomainException("Slug already
exists.", 409);
if (await db.OrderItems.AnyAsync(i => i.ProductId == id, ct))
    throw new DomainException("Product has historical orders and cannot be deleted. Deactivate it
instead.", 409);
```

What's happening: Slugify lowercases, splits on non-alphanumerics, and rejoins with dashes ('Banarasi Silk Saree' -> 'banarasi-silk-saree'); blank slugs derive from the name, explicit slugs are normalized the same way. Category existence and non-negative price/stock are checked before insert. Update preserves the existing slug when none is supplied. SKU auto-generates as NE-xxxxx. Delete guards run before FindAsync so conflicts report 409 even for otherwise-deletable rows.

Why it matters: The refuse-to-delete-history rule is what keeps old invoices resolvable forever — the database Restrict FKs would enforce it crudely; this enforces it politely.

Byte 70: AdminService: categories and order-status flow

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: Category CRUD, GetOrdersAsync, UpdateOrderStatusAsync

Builds on: Bytes 23, 27, 69

In plain terms: Categories get the same slug treatment with a populated-category delete guard; order status changes run through the PricingRules state machine before saving.

The code (csharp):

```
public async Task<bool> DeleteCategoryAsync(int id, CancellationToken ct = default)
{
    if (await db.Products.AnyAsync(p => p.CategoryId == id, ct))
        throw new DomainException("Category contains products and cannot be deleted.", 409);
    // ...
}
public async Task<OrderDto?> UpdateOrderStatusAsync(int id, string status, CancellationToken ct =
default)
{
    if (!PricingRules.ValidStatuses.Contains(status)) throw new DomainException("Invalid status.",
400);
    if (!PricingRules.IsValidTransition(o.Status, status)) throw new DomainException($"Cannot
transition from {o.Status} to {status}.", 409);
    o.Status = status;
```

What's happening: Category product counts come from a live query after save. UpdateOrderStatusAsync oddly constructs a throwaway OrderService (new OrderService(db, null!)) purely to reuse its private Map via GetDetailAsync — works, but a shared mapper would be cleaner. Status casing must match ValidStatuses exactly ('Placed', not 'placed').

Why it matters: Backward transitions 409 with a message naming both states — the admin UI surfaces this when a status change 'mysteriously' fails.

Byte 71: AdminEndpoints: the role-gated back office

Source: backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: AdminEndpoints.Map

Builds on: Bytes 35, 49, 68-70

In plain terms: Every admin route sits behind RequireAuthorization('Admin'): dashboard, product/category CRUD with 201/204 conventions, full order list, and status updates.

The code (csharp):

```
var g = app.MapGroup("/api/admin").WithTags("Admin").RequireAuthorization("Admin");
g.MapGet("/dashboard", async (IAdminService s) => Results.Ok(await s.GetDashboardAsync()));
g.MapDelete("/products/{id:int}", async (int id, IAdminService s) =>
{
    try { return await s.DeleteProductAsync(id) ? Results.NoContent() : Results.Problem("Product not
found.", statusCode: 404); }
    catch (DomainException e) { return Results.Problem(e.Message, statusCode: e.StatusCode); }
});
```

What's happening: The group policy means non-admins get 403 (authenticated) or 401 (anonymous) before any service code runs — verified by tests for all three cases. Product/category writes validate first, then map DomainException to 400/409. Status updates have no validator (any string reaches the service, which allow-lists). The pattern is deliberately repetitive: each route is independently readable.

Why it matters: Adding a new admin capability means one service method plus one grouped route here — the policy applies automatically.

PUTTING IT TOGETHER — Section C

PUTTING IT TOGETHER — Section C (part 2). Checkout validates, deduplicates by idempotency key, and commits order, stock, and cart-clear atomically; accounts keep exactly one default address; admins get live dashboards, slug-disciplined catalog CRUD with history-preserving delete guards, and a state-machine-gated order pipeline — all behind the Admin policy. The backend is fully drawn; Section D builds its mirror image in the browser: boot order, routing, headers, types, and formatting.

Section D — Frontend Foundation

Byte 72: package.json: scripts and the five dependencies

Source: frontend/package.json

Focus: Scripts and dependencies

Builds on: Byte 4

In plain terms: Five runtime libraries (Vue, Router, Pinia, Axios, Bootstrap) plus build/test tooling; npm scripts define the dev, build, typecheck, unit, and e2e workflows.

The code (json):

```
{ "scripts": { "dev": "vite",
    "build": "vue-tsc --noEmit && vite build",
    "test:unit": "vitest run",
    "test:e2e": "playwright test",
    "typecheck": "vue-tsc --noEmit" },
  "dependencies": { "axios": "^1.7.7", "bootstrap": "^5.3.3", "pinia": "^2.2.4",
    "vue": "^3.4.38", "vue-router": "^4.4.5" } }
```

What's happening: build typechecks before bundling — a type error fails production builds, not just editors. Vitest + Vue Test Utils + jsdom power component/store tests; Playwright drives real browsers. No UI kit beyond Bootstrap, no form library (hand-rolled validators), no state beyond Pinia.

Why it matters: Every CI/test command in the README resolves to one of these scripts; 'npm run build' passing means types plus bundle are both green.

Byte 73: vite.config.ts: dev server, proxy, and test setup

Source: frontend/vite.config.ts

Focus: Alias, server proxy, vitest coverage

Builds on: Byte 72

In plain terms: Vite serves the SPA on :5173 with /api proxied to the local backend, '@' aliased to src, and Vitest configured for jsdom component tests with v8 coverage.

The code (typescript):

```
resolve: { alias: { '@': path.resolve(__dirname, 'src') } },
server: { port: 5173,
  proxy: { '/api': { target: 'http://localhost:5000', changeOrigin: true } } },
test: { environment: 'jsdom', globals: true,
  include: ['src/**/*.{test,spec}.ts'], exclude: ['e2e/**', 'node_modules/**'] }
```

What's happening: The proxy is what makes local development work: the SPA calls relative '/api/...' and Vite forwards to the .NET backend on :5000, dodging CORS entirely. '@' alias matches tsconfig paths so imports stay short. Coverage includes all src TS/Vue except specs and .d.ts files — views included, which is why overall coverage reads low while stores read high.

Why it matters: A dev-time 'API unreachable' usually means the backend is not on :5000 — the proxy target here is the first thing to verify.

Byte 74: TypeScript strictness and module plumbing

Source: frontend/tsconfig.json; frontend/tsconfig.node.json; frontend/src/vite-env.d.ts

Focus: Compiler options and Vue declarations

Builds on: Byte 72

In plain terms: Strict mode with ES2020/DOM libs, bundler resolution, and '@/*' paths; a separate node config covers build files; a declaration shim lets TypeScript import .vue files.

The code (json):

```
{ "compilerOptions": { "target": "ES2020", "strict": true, "moduleResolution": "bundler",
    "paths": { "@/*": ["src/*"] } },
  "include": ["src/**/*.ts", "src/**/*.d.ts", "src/**/*.vue"] }
declare module '*.vue' { const component: DefineComponent<...>; export default component; }
```

What's happening: strict:true catches null/undefined misuse across stores and views (e.g. billing possibly null). noUnusedLocals/Parameters stay off to keep the port frictionless. tsconfig.node.json isolates

vite/playwright configs under Node types. The '*.vue' shim is what makes 'import App from @/App.vue' typecheck.

Why it matters: vue-tsc failures in CI almost always trace to nullable DTO fields (imageUrl?, line2?) — strict null checks doing their job.

Byte 75: index.html: the SPA mount point

Source: frontend/index.html

Focus: Root HTML shell

Builds on: Byte 1

In plain terms: A minimal HTML shell: mount div, app title, and the module script that boots Vue. Nginx serves this file for every non-file URL.

The code (html):

```
<div id="app"></div>
<script type="module" src="/src/main.ts"></script>
```

What's happening: Vite injects built assets at build time; the favicon references /vite.svg. Everything visible renders inside #app via main.ts mounting App.vue. Because Nginx falls back to this file (Byte 3), client-side routes resolve after load — the server never knows about /products/5.

Why it matters: White-screen-on-load bugs start here: a wrong script path or missing #app breaks the entire storefront at once.

Byte 76: main.ts: restore session before routing

Source: frontend/src/main.ts

Focus: App bootstrap with auth-first ordering

Builds on: Bytes 6, 49

In plain terms: Pinia installs first, then the auth store silently refreshes the session, and only afterward do the router and mount happen — so the first navigation's guards see the true login state.

The code (typescript):

```
const auth = useAuthStore(pinia)
auth
  .init()
  .catch(() => undefined)
  .finally(() => {
    app.use(router)
    app.mount('#app')
  })
```

What's happening: auth.init() calls POST /auth/refresh with cookies; success restores user+token, failure clears to guest — both resolve, never reject, so boot always completes. Installing the router inside .finally() is the documented fix for the auth-restore race: previously, full-page loads of /checkout bounced to /login

even with a valid session because guards ran before refresh finished. Plain promise chaining (no top-level await) honors the ES2020 build target.

Why it matters: This ordering is load-bearing: move app.use(router) above init and protected deep-links break on refresh.

Byte 77: App.vue: the persistent shell

Source: frontend/src/App.vue

Focus: Root component layout

Builds on: Byte 76

In plain terms: The root component is pure layout: header on top, routed page in the middle, footer below, plus two global overlays (mini-cart drawer and toast stack) that any view can trigger.

The code (vue):

```
<div class="d-flex flex-column min-vh-100">
  <AppHeader />
  <main class="container flex-grow-1 py-4">
    <RouterView />
  </main>
  <AppFooter />
  <CartDrawer />
  <NotificationToast />
</div>
```

What's happening: No script logic beyond imports — all state lives in Pinia. min-vh-100 plus flex-grow pins the footer to the viewport bottom on short pages. CartDrawer and NotificationToast are global because cart/UI stores are global; any component can open the drawer or toast without prop drilling.

Why it matters: New global UI (e.g. a promo banner) belongs here; page-specific UI belongs in views.

Byte 78: Router: the checkAccess decision table

Source: frontend/src/router/index.ts

Focus: checkAccess pure function

Builds on: Bytes 6, 49

In plain terms: One pure function decides every navigation: auth-required routes bounce guests to login (remembering where they went), guest-only routes bounce members home, admin routes additionally demand the Admin role.

The code (typescript):

```
export function checkAccess(to, auth): AccessCheck {
  if (to.meta.requiresAuth && !auth.isAuthenticated) {
    return { allowed: false, redirect: '/login', query: { returnUrl: to.fullPath } }
  }
  if (to.meta.requiresGuest && auth.isAuthenticated) {
    const ret = to.query?.returnUrl
```

```
    return { allowed: false, redirect: typeof ret === 'string' ? ret : '/' }
  }
  if (to.meta.requiresAdmin) {
    if (!auth.isAuthenticated) {
      return { allowed: false, redirect: '/login', query: { returnUrl: to.fullPath } }
    }
    if (!auth.isAdmin) {
      return { allowed: false, redirect: '/' }
    }
  }
  return { allowed: true }
}
```

What's happening: Pure (no router/store imports) so unit tests call it directly with fake route/auth objects — five specs cover every branch. returnUrl preservation is what sends a guest from /checkout to /login and back to /checkout after signing in. Admin checks nest auth-first so guests get login, not home.

Why it matters: Any 'I can see admin pages' or 'login loop' bug is decided here before any component loads.

Byte 79: Router: the fourteen routes

Source: frontend/src/router/index.ts

Focus: Route table

Builds on: Byte 78

In plain terms: Public catalog/cart pages are open; checkout, account, and order pages need login; three admin pages need the Admin role; product and order ids ride as route params; anything else is a 404 view.

The code (typescript):

```
{ path: '/products/:id', name: 'product-details',
  component: () => import('@/views/ProductDetailsView.vue'), props: true },
{ path: '/checkout', name: 'checkout',
  component: () => import('@/views/CheckoutView.vue'), meta: { requiresAuth: true } },
{ path: '/admin/products', name: 'admin-products',
  component: () => import('@/views/AdminProductsView.vue'),
  meta: { requiresAuth: true, requiresAdmin: true } },
{ path: '/:pathMatch(.*)*', name: 'not-found', component: () => import('@/views/NotFoundView.vue') }
```

What's happening: Lazy () => import() splits each view into its own chunk loaded on demand. props:true passes :id straight into ProductDetailsView/OrderDetailsView as a prop. History mode (createWebHistory) gives clean URLs that depend on the Nginx fallback (Byte 3). Route names power programmatic navigation (router.push('/...') and named links).

Why it matters: Deep-linking, code-splitting, and guard metadata all live in this one table — the frontend's equivalent of the backend's endpoint Map calls.

Byte 80: Router: the global guard

Source: frontend/src/router/index.ts

Focus: beforeEach navigation guard

Builds on: Bytes 78, 79

In plain terms: Before every navigation, the guard reads live auth state, runs the decision table, and redirects or allows — three lines that protect the whole app.

The code (typescript):

```
router.beforeEach((to) => {
  const auth = useAuthStore()
  const result = checkAccess(to, { isAuthenticated: auth.isAuthenticated, isAdmin: auth.isAdmin })
  if (!result.allowed && result.redirect) {
    return { path: result.redirect, query: result.query ?? {} }
  }
  return true
})
```

What's happening: useAuthStore() inside the guard reads the singleton state (populated by main.ts init before first navigation). Returning a path object redirects; returning true proceeds. Getters (isAuthenticated/isAdmin) are reactive, so post-login navigations re-evaluate with fresh state. No per-route guards exist — meta flags plus this one hook cover everything.

Why it matters: Because guards see restored state (Byte 75), refresh-on-/checkout keeps you on checkout instead of bouncing to login.

Byte 81: API client: guest id and token memory

Source: frontend/src/api/client.ts

Focus: Cart id helpers and access-token memory

Builds on: Bytes 6, 19

In plain terms: The guest cart id persists in localStorage (UUID when available); the access token lives only in a module variable — deliberately never in storage.

The code (typescript):

```
export function getCartId(): string {
  try {
    let id = localStorage.getItem(CART_KEY)
    if (!id) {
      id = newCartId()
      localStorage.setItem(CART_KEY, id)
    }
    return id
  } catch {
    return newCartId()
  }
}
// In-memory access token (never persisted to localStorage per requirements)
let accessTokenMemory = ''
```

What's happening: newCartId prefers crypto.randomUUID, falling back to a timestamp+random string; storage failures (private mode) degrade to an ephemeral id rather than crashing. CART_KEY 'cart_id' mirrors the backend cookie name conceptually but is separate storage. Keeping the token in memory

means XSS cannot harvest it from localStorage — at the cost of requiring a refresh call on every full page load.

Why it matters: This is the 'where trust lives' decision from the reflection notes: token in memory, session in HttpOnly cookie, guest identity in localStorage.

Byte 82: API client: every request carries auth + cart

Source: frontend/src/api/client.ts

Focus: Request interceptor

Builds on: Byte 81

In plain terms: An Axios interceptor attaches the Bearer token and X-Cart-Id to every outgoing call automatically — components and stores never set headers by hand.

The code (typescript):

```
apiClient.interceptors.request.use((config: InternalAxiosRequestConfig) => {
  if (accessTokenMemory) {
    config.headers = config.headers ?? ({} as never)
    ;(config.headers as Record<string, string>)['Authorization'] = `Bearer ${accessTokenMemory}`
  }
  try {
    const cartId = getCartId()
    if (cartId) {
      config.headers = config.headers ?? ({} as never)
      ;(config.headers as Record<string, string>)['X-Cart-Id'] = cartId
    }
  } catch {
    // ignore storage errors
  }
  return config
})
```

What's happening: baseURL '/api' with withCredentials:true means cookies (refresh) ride along on every call while tokens go in headers. The guest id is read fresh per request so post-login cart changes propagate without reloads. Leading-semicolon style guards against ASI hazards after the assignment lines.

Why it matters: One place sets headers for ~40 call sites; a missing Authorization header anywhere means this interceptor was bypassed (e.g. a raw axios call).

Byte 83: API client: silent refresh on 401, else clean logout

Source: frontend/src/api/client.ts

Focus: Response interceptor with _retry guard

Builds on: Bytes 47, 82

In plain terms: A 401 triggers one silent refresh attempt and retries the original request; if refresh fails, auth state clears and the browser is sent to login.

The code (typescript):

```
if (status === 401 && original && !original._retry && !url.includes('/auth/refresh')) {
  original._retry = true
  try {
    // Direct axios call (not apiClient) to avoid interceptor recursion
    const refreshRes = await axios.post('/api/auth/refresh', {}, { withCredentials: true })
    const data = refreshRes.data as { accessToken?: string; user?: unknown }
    if (data?.accessToken) {
      setAccessToken(data.accessToken)
      ;(original.headers as Record<string, string>)['Authorization'] = `Bearer ${data.accessToken}`
      return apiClient(original)
    }
  } catch {
    // fall through to logout handling
  }
```

What's happening: The _retry flag plus the /auth/refresh exclusion prevent infinite refresh loops. The refresh uses bare axios (not apiClient) so it cannot retrigger this interceptor. Only the token updates here — the user object refreshes on next fetchMe/init; endpoints key off the token anyway. Final fallback clears Pinia auth and hard-navigates to /login (deliberately crude: a dead session means a clean reload).

Why it matters: Expired-token UX (seamless retry vs login redirect) is entirely decided in these ~35 lines.

Byte 84: Types: user, auth, and catalog shapes

Source: frontend/src/types/index.ts

Focus: User, Auth, Category, Product interfaces

Builds on: Bytes 17, 18

In plain terms: TypeScript mirrors of the backend DTOs: users, auth envelopes, categories, and product cards with optional discount/stock extras.

The code (typescript):

```
export interface UserDto { id: string; email: string; firstName: string; lastName: string; role: string
}
export interface AuthResponse { accessToken: string; expiresAt: string; user: UserDto }
export interface ProductCardDto { id: string; name: string; slug: string;
  price: number; compareAtPrice?: number | null; discountPct?: number | null;
  stock: number; inStock: boolean; categoryId: string; categoryName?: string | null }
```

What's happening: Ids are strings here (backend ints serialize as JSON numbers; the SPA treats them opaquely — a deliberate looseness that avoids number/string friction in URLs). Optional/nullable marks (?, | null) mirror C# nullability so strict mode forces handling. ProductSort union pins sort values to the six backend-accepted keys. Note the drift: backend UserDto.Id is int; the frontend types it string — works because ids are only displayed and interpolated, never arithmetized.

Why it matters: These interfaces are the compile-time contract with the API; a backend DTO rename breaks the build here first — by design.

Byte 85: Types: cart, address, and order shapes

Source: frontend/src/types/index.ts

Focus: Cart, Address, Order interfaces and method unions

Builds on: Bytes 19-21

In plain terms: Cart lines plus totals, flexible addresses (ids optional for new/checkout snapshots), order receipts, and string unions that restrict shipping/payment choices at compile time.

The code (typescript):

```
export type ShippingMethod = 'standard' | 'express' | 'priority'
export type PaymentMethod = 'cod' | 'card_demo' | 'upi_demo' | 'netbanking_demo'
export interface CreateOrderRequest { shippingAddress: AddressDto; billingAddress?: AddressDto | null;
  same?: boolean; shippingMethod: ShippingMethod; paymentMethod: PaymentMethod; idempotencyKey?: string
}
```

What's happening: AddressDto.id is optional because new/checkout addresses have no id yet; isDefault/phone are optional for the same reason. CartDto mirrors the backend totals 1:1 (subTotal/shipping/tax/grandTotal/itemCount). The method unions duplicate PricingRules allow-lists — kept in sync by hand, a documented SPA+REST cost. OrderSummaryDto is the lightweight history row.

Why it matters: The CheckoutForm's selects bind to these unions, so an invalid method cannot even be selected, let alone sent.

Byte 86: Types: admin, toast, and error shapes

Source: frontend/src/types/index.ts

Focus: Admin upserts, ToastItem, ProblemDetails

Builds on: Bytes 21, 24

In plain terms: Admin forms get upsert shapes (create and edit share one form), toasts get a severity union, and backend errors get a ProblemDetails interface for message extraction.

The code (typescript):

```
export interface AdminProductUpsert { name: string; slug: string; price: number;
  compareAtPrice?: number | null; stock: number; categoryId: string;
  isActive: boolean; isFeatured: boolean }
export interface ToastItem { id: string; message: string; type: 'success' | 'danger' | 'info' |
'warning' }
export interface ProblemDetails { title?: string; detail?: string; status?: number;
  errors?: Record<string, string[]> }
```

What's happening: Upsert (not separate Create/Update) matches the shared AdminProductForm. ToastItem.type drives Bootstrap color classes in NotificationToast. ProblemDetails mirrors RFC 9457 (the backend's error format) so catch blocks can surface server messages instead of generic failures — though most views currently show fixed toast text.

Why it matters: Standardizing on these three shapes is what would let a future error-toast pass show field-level backend errors in forms.

Byte 87: format.ts: money, dates, image fallback

Source: frontend/src/utils/format.ts; frontend/public/placeholder.svg

Focus: formatINR, formatDate, onImageError

Builds on: Byte 18

In plain terms: One currency formatter (INR, India locale), one date formatter, and one image-error handler that swaps broken images for a local placeholder exactly once.

The code (typescript):

```
export function formatINR(value: number | null | undefined): string {
  const n = typeof value === 'number' && Number.isFinite(value) ? value : 0
  return new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' }).format(n)
}
export function onImageError(event: Event): void {
  const el = event.target as HTMLImageElement | null
  if (el && !el.dataset.fallback) {
    el.dataset.fallback = '1'
    el.src = PLACEHOLDER_IMAGE
  }
}
```

What's happening: Intl does locale grouping (12,999 with Indian digit grouping) plus the ₹ symbol — no manual string math. formatDate guards null/NaN into empty strings so tables never render 'Invalid Date'. The dataset.fallback flag stops infinite error loops if the placeholder itself fails. PLACEHOLDER_IMAGE points at /placeholder.svg served from Nginx static files — a 600x400 grey SVG reading 'No image' (frontend/public/placeholder.svg), so no code explanation is needed beyond its role as the final visual fallback.

Why it matters: Every price and date the shopper sees flows through these — consistent locale formatting with zero per-component effort.

Byte 88: validators.ts: client-side form gates

Source: frontend/src/utils/validators.ts

Focus: isRequired/isEmail/minLength and form validators

Builds on: Byte 39

In plain terms: Tiny predicate helpers plus per-form validators returning a {valid, errors} map that components render inline — fast feedback before any network call.

The code (typescript):

```
export function validateLoginForm(email: string, password: string): ValidationResult {
  const errors: Record<string, string> = {}
  if (!isRequired(email)) errors.email = 'Email is required'
```

```
  else if (!isEmail(email)) errors.email = 'Enter a valid email address'
  if (!isRequired(password)) errors.password = 'Password is required'
  else if (!minLength(password, 6)) errors.password = 'Password must be at least 6 characters'
  return { valid: Object.keys(errors).length === 0, errors }
}
```

What's happening: Client rules are intentionally looser than server rules (6-char minimum vs the server's 8+complexity) — they catch typos, the server enforces policy. validateRegisterForm and validateAddress follow the same errors-map shape; LoginForm/RegisterForm render errors[field] under each input. No dependency on any validation library keeps the bundle lean.

Why it matters: Duplicated validation is the known SPA+REST tax: UX speed here, truth on the server (Byte 39).

PUTTING IT TOGETHER — Section D

PUTTING IT TOGETHER — Section D. Strict TypeScript, proxied dev serving, and five small dependencies frame the app; auth-first boot feeds true state into pure route guards; one Axios client attaches identity to every call and heals expired tokens once; mirrored DTO types plus money/date/validation utilities keep display and input honest. Section E now spends this foundation across five stores, seventeen components, and fourteen views.

Section E — Frontend Features

Byte 89: auth store: session state and actions

Source: frontend/src/stores/auth.ts

Focus: useAuthStore state, getters, actions

Builds on: Bytes 49, 76, 81

In plain terms: The auth store holds the user plus access token, derives logged-in/admin flags, and wraps every auth API call — login, register, logout, silent refresh, and profile fetch.

The code (typescript):

```
getters: {
  isAuthenticated: (s): boolean => !!s.accessToken && !!s.user,
  isAdmin: (s): boolean => s.user?.role === 'Admin'
},
async login(email: string, password: string): Promise<UserDto> {
  const res = await apiClient.post<AuthResponse>('/auth/login', { email, password })
  this.applyAuth(res.data)
  return res.data.user
},
async refresh(): Promise<UserDto | null> {
  try {
    const res = await apiClient.post<AuthResponse>('/auth/refresh', {})
    this.applyAuth(res.data)
    return res.data.user
  } catch {
    this.clearAuth()
```

```
    return null
  }
},
```

What's happening: applyAuth writes both Pinia state and the apiClient token memory (Byte 80) together — they can never disagree. isAuthenticated requires both token and user, so a half-restored session still counts as logged out. logout and refresh swallow failures into clearAuth (fail-closed). init() is just refresh() renamed for main.ts boot. fetchMe re-syncs the profile after edits.

Why it matters: Guards (Byte 79), header buttons, and every protected view read these two getters; a wrong getter would unlock or lock the whole app.

Byte 90: cart store: basket state and operations

Source: frontend/src/stores/cart.ts

Focus: useCartStore state and CRUD actions

Builds on: Bytes 56, 81

In plain terms: The cart store mirrors the backend CartDto field-for-field and exposes fetch/add/update/remove/clear/merge — each syncing the full cart snapshot after the call.

The code (typescript):

```
function emptyState(): CartState {
  return { items: [], subTotal: 0, shipping: 0, tax: 0, grandTotal: 0, itemCount: 0 }
},
async add(productId: string, quantity = 1): Promise<void> {
  const res = await apiClient.post<CartDto>('/cart/items', { productId, quantity })
  this.setCart(res.data)
},
async mergeAfterLogin(): Promise<void> {
  // X-Cart-Id header is attached automatically by the api client
  const res = await apiClient.post<CartDto>('/cart/merge')
  this.setCart(res.data)
}
```

What's happening: setCart null-guards every total (dto.subTotal ?? 0) so a malformed response degrades to zeros, not NaN. clear() resets locally after DELETE (no response body to trust). mergeAfterLogin sends no body — identity comes from the Bearer token plus the automatic X-Cart-Id header. Getters (isEmpty, counts) drive empty-states and badges without recomputation.

Why it matters: Header badge, drawer, cart page, and checkout summary all render this one state — one fetch updates every surface at once.

Byte 91: catalog store: categories, search state, paging

Source: frontend/src/stores/catalog.ts

Focus: useCatalogStore filters and fetchers

Builds on: Bytes 52, 53, 81

In plain terms: The catalog store keeps categories plus the current product page and all active filters; fetchProducts merges caller overrides with stored filter state and syncs everything back from the response.

The code (typescript):

```
async fetchProducts(overrides?: { page?: number; pageSize?: number; search?: string;
    sort?: ProductSort; categoryId?: string }): Promise<void> {
  const page = overrides?.page ?? this.page
  const params: Record<string, string | number> = { page, pageSize }
  if (categoryId) params.categoryId = categoryId
  if (search) params.search = search
  if (sort) params.sort = sort
  const res = await apiClient.get<{ items: ProductCardDto[]; page: number;
    pageSize: number; totalCount: number; totalPages: number }>('/products', { params })
```

What's happening: Empty search/category are omitted from params (cleaner URLs, no backend behavior change). Response page/pageSize overwrite local asks — the server is authoritative on clamping (Byte 53). Plain setters (setSearch/setSort/setCategory/setPage) let views tweak filters without fetching; ProductListView drives fetches via route-query watches instead.

Why it matters: This is the single place where 'what the shopper asked' meets 'what the server answered'; paging display bugs live here.

Byte 92: orders store: idempotent placement and history

Source: frontend/src/stores/orders.ts

Focus: useOrdersStore and newIdempotencyKey

Builds on: Bytes 61, 81

In plain terms: Placing an order always attaches a fresh idempotency key unless the caller supplied one; history and detail fetchers keep the receipt views fed.

The code (typescript):

```
export function newIdempotencyKey(): string {
  return uuid()
},
async placeOrder(payload: CreateOrderRequest): Promise<OrderDto> {
  const body: CreateOrderRequest = {
    ...payload,
    idempotencyKey: payload.idempotencyKey || uuid()
  }
  const res = await apiClient.post<OrderDto>('/orders', body)
  this.current = res.data
  return res.data
}
```

What's happening: uuid() prefers crypto.randomUUID with a timestamp fallback — unique per click, which is exactly the granularity idempotency needs (retries reuse the key only if the caller holds it; separate clicks are separate orders). current holds the latest receipt for the confirmation screen; history holds the lightweight list. No totals math exists here — display only.

Why it matters: Every checkout click is born idempotent without the view thinking about it; double-submit protection starts here and lands in Byte 61.

Byte 93: ui store: loading, toasts, mini-cart

Source: frontend/src/stores/ui.ts

Focus: useUiStore state and notify/dismiss

Builds on: Byte 77

In plain terms: Three global UI concerns in one tiny store: a loading flag, a toast list with unique ids, and whether the mini-cart drawer is open.

The code (typescript):

```
notify(message: string, type: ToastItem['type'] = 'info'): string {
  const id = uid()
  this.toasts.push({ id, message, type })
  return id
},
dismiss(id: string) {
  this.toasts = this.toasts.filter((t) => t.id !== id)
},
```

What's happening: uid() combines timestamp + random so toast keys never collide. notify returns the id for programmatic dismissal; most callers fire-and-forget. No auto-expire exists — toasts persist until dismissed (a deliberate simplicity; NotificationToast renders them all). setMiniCartOpen is called after every successful add-to-cart for instant feedback.

Why it matters: Global feedback without prop drilling: any store or view can toast, and App.vue's NotificationToast renders it.

Byte 94: AppHeader: nav, search, cart badge, auth buttons

Source: frontend/src/components/AppHeader.vue

Focus: Header script and template

Builds on: Bytes 52, 89, 90, 93

In plain terms: The header loads categories and the cart on mount, links the first five categories, searches via route query, shows a live cart count badge, and swaps Account/Logout versus Login/Register by auth state.

The code (vue):

```
onMounted(async () => {
  try { await catalog.fetchCategories() } catch { /* ignore */ }
  try { await cart.fetch() } catch { /* ignore */ }
})
function search() {
  router.push({ path: '/products', query: { search: q.value || undefined } })
}
```

```
async function doLogout() {
  await auth.logout()
  await cart.fetch().catch(() => undefined)
  router.push('/')
}
```

What's happening: Mount failures are swallowed so a dead API still renders navigation (degraded, not blank). Search pushes a query URL — ProductListView's watcher does the actual fetching. The badge (data-testid='cart-count') binds cart.itemCount reactively. Logout refreshes the cart because ownership changed (user cart -> fresh guest view) and routes home. Admin link renders only for admins.

Why it matters: This is the only component that touches auth, cart, catalog, and UI stores together — the app's integration smoke test in miniature.

Byte 95: AppFooter and NotFoundView: chrome and 404

Source: frontend/src/components/AppFooter.vue; frontend/src/views/NotFoundView.vue

Focus: Full file contents (both trivial)

Builds on: Byte 77

In plain terms: The footer is three static links with a copyright line; the catch-all route renders a centered 404 with a home button. No logic in either.

The code (vue):

```
<footer class="bg-light border-top mt-5 py-4">
  <span class="text-muted">\u00a9 2026 Yantra Shop. All rights reserved.</span>
  <RouterLink class="me-3" to="/products">Products</RouterLink>
</footer>
<h1 class="display-4">404</h1>
<p class="lead">Page not found.</p>
<RouterLink class="btn btn-primary" to="/">Go home</RouterLink>
```

What's happening: Both are stateless templates with no script logic (empty <script setup> blocks). The 404 only triggers for URLs that miss every route AND miss Nginx static files — API 404s never reach it. Footer links use RouterLink so they stay inside the SPA.

Why it matters: Grouped as one byte because neither contains behavior worth a solo explanation; they complete the layout contract from Byte 77.

Byte 96: ProductCard and ProductList: the catalog grid

Source: frontend/src/components/ProductCard.vue; frontend/src/components/ProductList.vue

Focus: Card and grid templates

Builds on: Bytes 84, 87

In plain terms: ProductCard renders one product tile (image, name, price, sale badge, stock badge) linking to details; ProductList grids the cards and shows an empty message when there is nothing to show.

The code (vue):

```
<RouterLink :to="`/products/${product.id}`" :aria-label="`View ${product.name}`">
  <img :src="product.imageUrl || PLACEHOLDER_IMAGE" :alt="product.name"
    class="card-img-top" loading="lazy" @error="onImageError" />
</RouterLink>
<strong>{{ formatINR(product.price) }}</strong>
<span v-if="product.discountPct" class="badge bg-success">{{ product.discountPct }}% off</span>
<span v-if="!product.inStock" class="badge bg-danger mt-2 align-self-start">Out of stock</span>
```

What's happening: Images lazy-load with error fallback (Byte 87); missing URLs fall back before even erroring. Prices format via formatINR; compare-at prices strike through only when present. data-testid hooks ('product-card', 'product-list') exist for E2E selectors. ProductList is a pure presentational loop (v-for with :key) over a products prop — HomeView and ProductListView share it.

Why it matters: Every product the shopper ever sees renders through this card; a display bug here is a store-wide display bug.

Byte 97: ProductDetails: buy box with quantity

Source: frontend/src/components/ProductDetails.vue

Focus: addToCart and buy-box template

Builds on: Bytes 84, 87, 90

In plain terms: The buy box shows price, stock status, a capped quantity input, and an Add button that adds to cart, toasts success, and pops the mini-cart open.

The code (vue):

```
const qty = ref(1)
async function addToCart() {
  try {
    await cart.add(props.product.id, qty.value)
    ui.notify('Added to cart', 'success')
    ui.setMiniCartOpen(true)
  } catch {
    ui.notify('Failed to add to cart', 'danger')
  }
}
<button class="btn btn-primary" :disabled="!product.inStock" type="button" @click="addToCart">
```

What's happening: Quantity caps at live stock (max=stock||99) so shoppers cannot request the impossible; the server still re-checks (Byte 57). The button disables for out-of-stock items. Failure toasts stay generic here — the server's specific reason (409 'only N available') is intentionally not surfaced, a UX gap worth knowing. data-testid='product-details' anchors E2E.

Why it matters: This is the cart-entry funnel: most cart contents originate from this one button.

Byte 98: CartDrawer: the mini-cart overlay

Source: frontend/src/components/CartDrawer.vue

Focus: Drawer visibility and content

Builds on: Bytes 77, 90

In plain terms: A slide-in panel bound to the ui store's miniCartOpen flag lists cart lines with the subtotal and shortcuts to the cart and checkout pages.

The code (vue):

```
<div class="offcanvas offcanvas-end" :class="{ show: ui.miniCartOpen }"
  :style="{ visibility: ui.miniCartOpen ? 'visible' : 'hidden' }"
  tabindex="-1" aria-label="Mini cart" data-testid="cart-drawer">
  <CartItemRow v-for="item in cart.items" :key="item.productId" :item="item" />
  <RouterLink class="btn btn-primary" to="/cart" @click="close">View cart</RouterLink>
```

What's happening: Visibility toggles via class + inline style (not Bootstrap JS) so Vue state is the single driver — no jQuery-style DOM fighting. Links close the drawer on navigation. Empty state is a muted line, not an error. Rendered globally in App.vue, it works from any page without mounting.

Why it matters: Instant purchase feedback (drawer pops on every add) is what makes the store feel fast despite full server round-trips.

Byte 99: CartItemRow: quantity edit and remove

Source: frontend/src/components/CartItemRow.vue

Focus: setQty and remove actions

Builds on: Bytes 87, 90

In plain terms: Each cart line shows the thumbnail, unit price, an editable quantity, and a remove button — quantity zero removes via the same path as delete.

The code (vue):

```
async function setQty(qty: number) {
  try {
    if (qty <= 0) { await cart.remove(props.item.productId) }
    else { await cart.update(props.item.productId, qty) }
  } catch {
    ui.notify('Failed to update cart', 'danger')
  }
}
```

What's happening: The input's max binds stockAvailable so the UI caps before the server must; @change (not @input) avoids a request per keystroke. Failures toast generically — stock-shrink rejections ('only N available') do not refresh the displayed max until next fetch, a minor staleness to know about. Used identically in the drawer and the cart page.

Why it matters: One row component serves both cart surfaces, so quantity behavior cannot drift between them.

Byte 100: Pagination, LoadingSpinner, NotificationToast: micro-chrome

Source: frontend/src/components/Pagination.vue; frontend/src/components/LoadingSpinner.vue; frontend/src/components/NotificationToast.vue

Focus: pages(), spinner, toast loop

Builds on: Bytes 91, 93

In plain terms: Pagination shows a five-button sliding window with guarded prev/next; the spinner is an accessible loading label; toasts render the ui store's list with severity colors and dismiss buttons.

The code (vue):

```
function pages(): number[] {
  const total = Math.max(1, props.totalPages)
  const current = Math.min(Math.max(1, props.page), total)
  const start = Math.max(1, current - 2)
  const end = Math.min(total, start + 4)
  const out: number[] = []
  for (let i = start; i <= end; i++) out.push(i)
  return out
}
```

What's happening: The window clamps at both ends (page 1 of 20 shows 1–5, page 19 shows 16–20); prev/next disable at the edges and emit 'change' with the target page — the parent (ProductListView) owns navigation. The spinner uses role=status + aria-live for screen readers. Toasts map type to Bootstrap text-bg classes and emit dismiss per id.

Why it matters: Three zero-dependency components grouped in one byte; together they standardize loading, paging, and feedback across every view.

Byte 101: LoginForm and RegisterForm: validated auth inputs

Source: frontend/src/components/LoginForm.vue; frontend/src/components/RegisterForm.vue

Focus: onSubmit validation-and-emit pattern

Builds on: Byte 88

In plain terms: Both forms validate locally with the shared validators, show inline field errors, and emit a submit payload only when valid — parent views own the actual API calls.

The code (vue):

```
function onSubmit() {
  const result = validateLoginForm(form.email, form.password)
  errors.value = result.errors
  if (result.valid) emit('submit', { ...form })
}
<button class="btn btn-primary w-100" type="submit" :disabled="props.loading">
  {{ props.loading ? 'Signing in\u2026' : 'Login' }}
</button>
```

What's happening: novalidate disables browser bubbles so custom messages rule. v-model binds inputs; is-invalid classes + invalid-feedback divs render errors[field]. The loading prop swaps button text and disables double-submit during the request. RegisterForm adds first/last name fields with the same pattern. data-testids ('login-form', 'register-form') anchor E2E fills.

Why it matters: Dumb-form/smart-view split: these components are unit-testable without any API mocking (and are).

Byte 102: AddressForm: generic two-way address editor

Source: frontend/src/components/AddressForm.vue

Focus: v-model update pattern with prefix

Builds on: Byte 85

In plain terms: A reusable address editor that emits a full updated object on every keystroke and prefixes element ids so multiple instances coexist on one page.

The code (vue):

```
function update<K extends keyof AddressDto>(key: K, value: AddressDto[K]) {
  emit('update:modelValue', { ...props.modelValue, [key]: value })
}
<input :id="`${prefix}-fullName`" :value="modelValue.fullName" class="form-control"
  @input="update('fullName', ($event.target as HTMLInputElement).value)" />
```

What's happening: Controlled-input pattern (:value + @input, not v-model) keeps the parent the source of truth — required for v-model composition in CheckoutForm and AccountDashboardView. The generic update preserves type safety across string/nullable fields. prefix ('ship'/'bill'/'acct') keeps label-for associations unique when shipping and billing forms share a page — E2E fills #ship-line1 precisely because of this.

Why it matters: One form serves checkout shipping, checkout billing, and address-book editing; address UX cannot diverge.

Byte 103: CheckoutForm: addresses plus method picks

Source: frontend/src/components/CheckoutForm.vue

Focus: Same-as-shipping state and submit payload

Builds on: Bytes 85, 102

In plain terms: CheckoutForm wraps two AddressForms with a 'same as shipping' toggle plus shipping/payment dropdowns, emitting one payload — billing null when same, a copy otherwise.

The code (vue):

```
const state = reactive({
  shipping: { ...props.shipping },
  billing: { ...(props.billing ?? { ...props.shipping }) },
  same: props.same,
  shippingMethod: props.shippingMethod as ShippingMethod,
  paymentMethod: props.paymentMethod as PaymentMethod
})
function submit() {
  emit('submit', { shipping: { ...state.shipping },
    billing: state.same ? null : { ...state.billing }, ... })
}
```

What's happening: Local reactive copies let shoppers edit freely without mutating parent state until submit. Unchecking 'same' reveals the billing form prefilled as a shipping copy (convenient starting point).

The <slot name='summary'> lets CheckoutView inject totals inside the form visually while OrderSummary stays a separate component. Method defaults (standard/cod) match backend defaults.

Why it matters: The same-flag contract matters downstream: CheckoutView converts null billing into a shipping copy because the API rejects null (Byte 40).

Byte 104: OrderSummary: receipt preview from cart state

Source: frontend/src/components/OrderSummary.vue

Focus: Full file (read-only cart render)

Builds on: Bytes 87, 90

In plain terms: A Bootstrap card listing each cart line plus subtotal, shipping, tax, and total — pure display bound straight to the cart store, with no inputs and no logic.

The code (vue):

```
<div v-for="item in cart.items" :key="item.productId" class="d-flex justify-content-between small">
  <span>{{ item.productName }} \u00d7 {{ item.quantity }}</span>
  <span>{{ formatINR(item.lineTotal) }}</span>
</div>
<div class="d-flex justify-content-between"><span>Shipping</span><span>{{ formatINR(cart.shipping)
}}</span></div>
```

What's happening: No props, no emits, no fetches — it renders whatever the cart store holds, so CheckoutView just drops it beside the form. data-testid='order-summary' anchors E2E total assertions. Because totals are server-computed (Byte 56), this preview always matches what checkout will charge (modulo the shipping-method difference).

Why it matters: Read-only receipt widgets cannot cause charge bugs; that is the point of the design.

Byte 105: AdminProductForm: shared create/edit form

Source: frontend/src/components/AdminProductForm.vue

Focus: Reactive draft, watch sync, submit/cancel

Builds on: Byte 86

In plain terms: One form serves product create and edit: a reactive draft syncs from the parent's modelValue, category picks from a dropdown, and submit emits the draft upward for the view to POST or PUT.

The code (vue):

```
const form = reactive<AdminProductUpsert>({ ...props.modelValue })
watch(() => props.modelValue, (v) => Object.assign(form, v), { deep: true })
function submit() {
  emit('update:modelValue', { ...form })
  emit('submit')
}
```

What's happening: The deep watcher re-syncs when the parent swaps drafts (create -> edit another product) without remounting. v-model.number keeps price/stock numeric for the API. Required attributes give first-line HTML validation; server 400s surface as toasts in the view. Slug is a free-text field (blank = auto-slug server-side). data-testid='admin-product-form' anchors the E2E create-product flow.

Why it matters: Shared create/edit halves admin form maintenance — and halves the places product validation display can break.

Byte 106: AdminOrderTable: status board with inline updates

Source: frontend/src/components/AdminOrderTable.vue

Focus: Row render and update-status emit

Builds on: Bytes 86, 87

In plain terms: A striped table of orders with formatted dates/totals and a per-row status dropdown that emits {id, status} upward — the view performs the actual update call.

The code (vue):

```
const statuses = ['Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled']
<select :id="`status-${o.id}`" :value="o.status" class="form-select form-select-sm"
  @change="emit('update-status', { id: o.id, status: ($event.target as HTMLSelectElement).value })">
```

What's happening: AdminOrderRow is a local display interface (id/orderNumber/date/status/total/itemCount). The dropdown lists a UI subset of backend statuses (no Delivered-mapped 'Completed'/'Refunded' options — admins cannot issue those states from this UI, a small functional gap). :value (not v-model) keeps the server the source of truth; the parent reloads after update. data-testid='admin-order-table' anchors E2E.

Why it matters: Dumb-table/smart-view again: the table is unit-testable with static rows, and status-flow rules stay server-side (Byte 70).

PUTTING IT TOGETHER — Section E

PUTTING IT TOGETHER — Section E (part 1). Stores own session, basket, search, receipts, and feedback; dumb components render while smart views fetch and mutate; the header integrates all four stores; checkout translates UI convenience into API contract; login and register preserve guest baskets across the identity boundary. Admin views plus deployment complete the tour next.

Section E — Frontend Features (views)

Byte 107: HomeView: hero, categories, featured rail

Source: frontend/src/views/HomeView.vue

Focus: onMounted data load and sections

Builds on: Bytes 91, 94, 96

In plain terms: The landing page fetches categories plus eight 'recommended' products on mount and renders a hero banner, a category grid with counts, and the shared product grid.

The code (vue):

```
onMounted(async () => {
  loading.value = true
  try {
    await catalog.fetchCategories()
    const res = await apiClient.get<{ items: ProductCardDto[] }>('/products', {
      params: { page: 1, pageSize: 8, sort: 'recommended' }
    })
    featured.value = res.data.items ?? []
  } catch {
    featured.value = []
  } finally {
    loading.value = false
  }
})
```

What's happening: Featured bypasses the catalog store (local ref) because it is display-only, while categories go through the store for header reuse. Failures degrade to empty sections, never a blank page. Category cards link to /products?categoryId= — the same query contract ProductListView reads. LoadingSpinner covers the fetch window.

Why it matters: The home page is the store's first impression and its cheapest page: two GETs, no auth, fully degradable.

Byte 108: ProductListView: filter state lives in the URL

Source: frontend/src/views/ProductListView.vue

Focus: syncFromQuery, load, pushQuery, query watcher

Builds on: Bytes 79, 91, 96, 100

In plain terms: Search, category, sort, and page live in the URL query string: the view syncs controls from the query, pushes control changes back into the query, and refetches whenever the query changes.

The code (vue):

```
function pushQuery(page?: number) {
  router.push({ path: '/products',
    query: { search: filters.search || undefined, categoryId: filters.categoryId || undefined,
      sort: filters.sort, page: String(page ?? catalog.page) } })
}
watch(() => route.query, () => { void load() })
```

What's happening: Empty filters become undefined so they vanish from the URL (shareable, bookmarkable links). Enter in search and change on selects reset to page 1 (new filters invalidate old pages). The watcher makes back/forward buttons and header searches work without extra code. Result count + page indicator render from store metadata; Pagination emits page targets into pushQuery.

Why it matters: URL-as-state is what makes search results linkable and the back button sane — the alternative (local-only state) breaks both.

Byte 109: ProductDetailsView: fetch-by-id with states

Source: frontend/src/views/ProductDetailsView.vue

Focus: onMounted fetch with loading/error

Builds on: Bytes 79, 97

In plain terms: The details page reads :id from the route, fetches that product, and renders exactly one of three states: spinner, error alert, or the buy box.

The code (vue):

```
onMounted(async () => {
  loading.value = true
  try {
    const res = await apiClient.get<ProductDetailDto>(`/products/${props.id}`)
    product.value = res.data
  } catch {
    error.value = 'Product not found.'
  } finally {
    loading.value = false
  }
})
```

What's happening: props:true (Byte 79) delivers the id with no $route plumbing. Any failure (404 inactive/missing, network) collapses to one message — simple but it hides retryable errors from shoppers. The view owns fetching while ProductDetails owns display and cart actions (dumb/smart split again).

Why it matters: Three-state (loading/error/content) rendering is the pattern every fetch view follows; learn it here, recognize it everywhere.

Byte 110: CartView: basket page with totals card

Source: frontend/src/views/CartView.vue

Focus: Fetch on mount and totals layout

Builds on: Bytes 90, 99

In plain terms: The cart page fetches the basket on mount, shows an empty alert or the line list beside a totals card with a checkout button — reusing the same row component as the drawer.

The code (vue):

```
<div v-else class="row g-4">
  <div class="col-md-8 d-flex flex-column gap-2">
    <CartItemRow v-for="item in cart.items" :key="item.productId" :item="item" />
  </div>
  <div class="col-md-4">
    <div class="card"><div class="card-body">
      <div class="d-flex justify-content-between"><span>Total</span><strong>{{
formatINR(cart.grandTotal) }}</strong></div>
      <RouterLink class="btn btn-success w-100 mt-3" to="/checkout">Proceed to checkout</RouterLink>
```

What's happening: Fetch failures are swallowed (stale or empty cart renders rather than an error wall). Totals mirror the drawer's numbers from the same store — consistency for free. The checkout link routes to the guarded /checkout, so guests hit the login flow with returnUrl (Byte 79) before paying.

Why it matters: Two cart surfaces, one store, one row component: quantity edits here and in the drawer can never disagree.

Byte 111: CheckoutView: place order and confirm

Source: frontend/src/views/CheckoutView.vue

Focus: place() with billing-copy fix and confirmation

Builds on: Bytes 61, 90, 92, 103

In plain terms: Checkout pairs the form with the live order summary; placing sends an explicit billing copy when 'same as shipping' is set (the E2E-caught fix), then shows a confirmation with the order number.

The code (vue):

```
const order = await orders.placeOrder({
  shippingAddress: payload.shipping,
  // Backend requires an explicit billing snapshot; "same as shipping"
  // is a UI convenience, so send a copy of the shipping address.
  billingAddress: payload.billing ?? payload.shipping,
  same: payload.same,
  shippingMethod: payload.shippingMethod,
  paymentMethod: payload.paymentMethod
})
placed.value = order
```

What's happening: The ?? fallback is the documented bug fix: CheckoutForm emits null billing when 'same' is checked, but the API 400s null billing — the view translates UI convenience into API contract. After placement the cart refetches (now empty) and a success toast names the order number. Confirmation (data-testid='order-confirmation') offers a details link; 'Placing order…' text covers the wait. Failures toast generically.

Why it matters: Read the comment in the code: it records a real cross-layer contract lesson — UI nullability vs API requiredness.

Byte 112: LoginView and RegisterView: authenticate then merge

Source: frontend/src/views/LoginView.vue; frontend/src/views/RegisterView.vue

Focus: onSubmit flows with mergeAfterLogin

Builds on: Bytes 79, 89, 90, 101

In plain terms: Both views submit their form to the auth store, fold the guest cart into the new session, refresh the cart view, and return the shopper to their original page.

The code (vue):

```
await auth.login(payload.email, payload.password)
await cart.mergeAfterLogin().catch(() => undefined)
await cart.fetch().catch(() => undefined)
const ret = typeof route.query.returnUrl === 'string' ? route.query.returnUrl : '/'
router.push(ret)
```

What's happening: Merge failures are non-fatal (guest had nothing worth keeping, or stock capped it) — login still succeeds. RegisterView is identical with auth.register, so new shoppers keep pre-signup baskets too. returnUrl (Byte 79) completes the guarded-checkout journey. Error toasts stay generic to avoid user enumeration ('Email may already be in use' is the only hint).

Why it matters: This four-line sequence is the guest-to-customer handoff from Byte 7 made real — login that lost your cart would feel like data loss.

Byte 113: AccountDashboardView: profile plus address book

Source: frontend/src/views/AccountDashboardView.vue

Focus: Profile save and address draft/edit flows

Builds on: Bytes 66, 67, 102

In plain terms: The account page edits first/last name in place and manages the address book through a draft object that doubles as the create form and the edit form.

The code (vue):

```
async function saveAddress() {
  try {
    if (editing.value?.id) {
      await apiClient.put(`/account/addresses/${editing.value.id}`, draft.value)
    } else {
      await apiClient.post('/account/addresses', draft.value)
    }
    ui.notify('Address saved', 'success')
    editing.value = null
    draft.value = { ...emptyAddress }
    await loadAddresses()
  } catch {
    ui.notify('Failed to save address', 'danger')
  }
}
```

What's happening: Profile prefill comes from the auth store user (populated at login/init); after save, fetchMe re-syncs the store so the header stays current. editing=null means 'creating'; startEdit clones the row into the draft so cancels never corrupt the list. Every mutation reloads the list from the server (source of truth for the default flag). Delete by id; the backend promotes a new default automatically.

Why it matters: Draft-copy editing (never mutate list rows directly) is what makes cancel safe — a pattern worth copying in admin forms.

Byte 114: OrderHistoryView and OrderDetailsView: past purchases

Source: frontend/src/views/OrderHistoryView.vue; frontend/src/views/OrderDetailsView.vue

Focus: fetchHistory/fetchDetail with table and receipt

Builds on: Bytes 65, 87, 92

In plain terms: History lists every order newest-first with links; details shows one order's lines and total or a not-found message — both thin renders over the orders store.

The code (vue):

```
onMounted(() => { orders.fetchHistory().catch(() => undefined) })
<td><RouterLink :to="`/account/orders/${o.id}`">{{ o.orderNumber }}</RouterLink></td>
onMounted(async () => {
  try { await orders.fetchDetail(props.id) }
  catch { error.value = 'Order not found.' }
})
```

What's happening: History's empty state ('No orders yet.') distinguishes new customers from load failures only weakly — fetch errors also render empty. Details reuses orders.current (shared with checkout's receipt), so navigating history->detail overwrites the confirmation receipt — harmless, since confirmation already displayed. 403s (other customers' orders) collapse into 'not found', matching the backend's opaqueness.

Why it matters: Two views, zero local state beyond one error string: the orders store does all the work.

Byte 115: AdminDashboardView: live counts and recent orders

Source: frontend/src/views/AdminDashboardView.vue

Focus: Dashboard + recent-orders loads

Builds on: Bytes 68, 87

In plain terms: The admin landing fetches dashboard aggregates and the five most recent orders in parallel-ish sequence, rendering stat cards with a management shortcut row.

The code (vue):

```
const res = await apiClient.get<AdminDashboardDto>('/admin/dashboard')
stats.value = res.data
// ...
const res = await apiClient.get<OrderSummaryDto[]>('/admin/orders')
recent.value = (res.data ?? []).slice(0, 5)
```

What's happening: Two independent try/catch blocks so a failing orders call cannot blank the stats (partial degradation again). Slicing client-side keeps the backend list endpoint simple. data-testid='admin-stats' anchors the E2E admin assertion. Non-admins never see this — the router (Byte 79) and the API policy (Byte 71) both gate it.

Why it matters: Back-office proof of life: if these numbers render, auth, policy, services, and database all work end to end.

Byte 116: AdminProductsView: product grid with create/edit

Source: frontend/src/views/AdminProductsView.vue

Focus: load, startCreate/startEdit, save, remove

Builds on: Bytes 69, 86, 105

In plain terms: Admin products load the grid plus categories together, prefill a shared draft for create or edit, save via POST/PUT, and delete behind a confirm dialog with list reloads after every mutation.

The code (vue):

```
async function load() {
  const [pRes, cRes] = await Promise.all([
    apiClient.get<{ items: ProductCardDto[] }>('/admin/products', { params: { page: 1, pageSize: 50 }
}),
    apiClient.get<CategoryDto[]>('/admin/categories')
  ])
  const items = (pRes.data as unknown as { items?: ProductCardDto[] }).items
  products.value = items ?? (pRes.data as unknown as ProductCardDto[])
}
async function remove(id: string) {
  if (!confirm('Delete this product?')) return
  try {
    await apiClient.delete(`/admin/products/${id}`)
    ui.notify('Product deleted', 'success')
    await load()
  } catch {
    ui.notify('Failed to delete product', 'danger')
  }
}
```

What's happening: Promise.all halves load latency. The items-??-array fallback tolerates envelope-vs-array response shapes (defensive against backend changes). startEdit maps the card DTO onto the upsert shape (isActive/isFeatured reset to safe defaults — note: editing an inactive product shows Active checked, a minor prefill bug). Delete failures (409 history-guard) surface as a generic toast, hiding the 'deactivate instead' guidance — admin UX gap. data-testid='admin-products-table' anchors E2E create assertions.

Why it matters: The full admin product lifecycle (the E2E 'create product' test) runs through this view's four functions.

PUTTING IT TOGETHER — Section E

PUTTING IT TOGETHER — Section E (part 2). The fourteen views turn stores into pages: URL-driven product discovery, three-state fetching, a shared cart surface, a billing-copy checkout with confirmation, login flows that preserve guest baskets, an address-book account page, plain order history, and two admin consoles for catalog and pipeline. Every view is thin by design — state in stores, headers in the client, rules on the server. The admin orders board and deployment close the tour next.

Section E — Frontend Features (admin orders)

Byte 117: AdminOrdersView: order pipeline board

Source: frontend/src/views/AdminOrdersView.vue

Focus: load and updateStatus

Builds on: Bytes 71, 93, 106

In plain terms: The admin orders page loads every order into the shared table component and pushes status changes through the admin endpoint, reloading after each update.

The code (vue):

```
async function updateStatus(payload: { id: string; status: string }) {
  try {
    await apiClient.put(`/admin/orders/${payload.id}/status`, { status: payload.status })
    ui.notify('Order status updated', 'success')
    await load()
  } catch {
    ui.notify('Failed to update status', 'danger')
  }
}
```

What's happening: The view is fourteen lines of glue: AdminOrderTable renders + emits, this function PUTs + reloads. Backward-transition 409s surface as the generic failure toast (the 'cannot transition' detail is lost — same admin-UX gap as product delete). Load failures toast once on mount. No local order state — the ref array is the whole model.

Why it matters: The order-status flow test (Placed->Delivered jump, backward rejection) exercises exactly this function against the real state machine.

PUTTING IT TOGETHER — Section E

PUTTING IT TOGETHER — Section E (part 3). The admin orders board closes the frontend feature loop: every view in the app is now a thin render over a store plus the shared api client. Section F follows the code into production: containers, images, proxying, caching, and the ignore-rules that protect secrets.

Section F — Infrastructure & Deployment

Byte 118: Compose db: SQL Server with a named volume

Source: docker-compose.yml

Focus: db service block

Builds on: Byte 5

In plain terms: SQL Server 2022 Developer runs with the EULA pre-accepted, the SA password injected from .env, data on a named volume, and a sqlcmd healthcheck that gates everything downstream.

The code (yaml):

```
db:
  image: mcr.microsoft.com/mssql/server:2022-latest
  environment:
    ACCEPT_EULA: "Y"
```

```
    MSSQL_SA_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD in .env}
    MSSQL_PID: Developer
  volumes:
    - portecommerce_mssql_data:/var/opt/mssql
  healthcheck:
    test: ["CMD-SHELL", "/opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -P \"$${MSSQL_SA_PASSWORD}\"
-C -Q 'SELECT 1' -b -l 5 || exit 1"]
    interval: 10s
    timeout: 5s
    start_period: 40s
    retries: 12
```

What's happening: No ports are published: the database is reachable only as hostname 'db' inside the compose network (the API connection string uses Server=db,1433). The fail-fast ${...:?} aborts compose with a clear message when .env is missing. start_period 40s honors SQL Server's slow cold start; 12 retries span ~2 minutes. The named volume survives 'compose down' — only 'down -v' destroys data (the README warns NEVER to do that).

Why it matters: Persistent volume + health-gated dependents is why restarts keep every order and why the API never boots against a half-ready database.

Byte 119: Compose api: the backend container

Source: docker-compose.yml

Focus: api service block

Builds on: Bytes 5, 118

In plain terms: The API builds from its Dockerfile, waits for a healthy database, receives its connection string and JWT settings from the environment, and proves health via the database health endpoint.

The code (yaml):

```
api:
  build: { context: ., dockerfile: backend/Ecommerce.Api/Dockerfile }
  depends_on: { db: { condition: service_healthy } }
  environment:
    ASPNETCORE_ENVIRONMENT: Production
    ConnectionStrings__Default: "Server=db,1433;Database=EcommerceDb;User
Id=sa;Password=${DB_PASSWORD:?set DB_PASSWORD in .env};TrustServerCertificate=True"
    JWT_KEY: ${JWT_KEY:?set JWT_KEY in .env}
    JWT_ISSUER: ecommerce-api
    JWT_AUDIENCE: ecommerce-spa
  expose: ["8080"]
  healthcheck:
    test: ["CMD-SHELL", "curl -fsS http://localhost:8080/health/db || exit 1"]
```

What's happening: Double-underscore ConnectionStrings__Default maps to the .NET config key ConnectionStrings:Default. TrustServerCertificate=True accepts the container SQL Server's self-signed cert (dev convenience; production should use proper certs). expose (not ports) keeps the API off the host network — only Nginx reaches it. The healthcheck hits /health/db (not bare /health) so Nginx only routes when the database is actually queryable. start_period 120s covers migration+seed time.

Why it matters: Every production 401/500-after-deploy traces through these six env lines first — key divergence and connection strings live here.

Byte 120: Compose nginx, network, volume: the front door

Source: docker-compose.yml

Focus: nginx service, networks, volumes

Builds on: Bytes 2, 3, 5

In plain terms: Nginx builds the SPA and serves it on the host port, starts only after a healthy API, and all three services share one isolated bridge network with one named data volume.

The code (yaml):

```
nginx:
  build: { context: ., dockerfile: nginx/Dockerfile }
  depends_on: { api: { condition: service_healthy } }
  ports: ["${HOST_PORT:-8080}:80"]
networks:
  portecommerce_net: { name: portecommerce_net, driver: bridge }
volumes:
  portecommerce_mssql_data: { name: portecommerce_mssql_data }
```

What's happening: ports publishes only Nginx (host:container), defaulting to 8080 with HOST_PORT override. Named network/volume (explicit names, not compose-generated hashes) isolate this stack from the legacy Week-3 project so both run side by side (legacy :8085, this :8080). The top-level 'name: portecommerce' prefixes any generated resource. Nginx's own wget healthcheck proves it serves before compose reports healthy.

Why it matters: Isolation-by-naming is the coexistence contract with the legacy stack — rename these and the two projects collide.

Byte 121: Backend Dockerfile: SDK build, runtime ship

Source: backend/Ecommerce.Api/Dockerfile

Focus: Multi-stage build

Builds on: Bytes 10, 119

In plain terms: A two-stage build restores and publishes the API with the .NET 10 SDK, then ships only the publish output on the lean ASP.NET runtime with curl added for healthchecks.

The code (dockerfile):

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY backend/Ecommerce.Core/Ecommerce.Core.csproj backend/Ecommerce.Core/
COPY backend/Ecommerce.Infrastructure/Ecommerce.Infrastructure.csproj backend/Ecommerce.Infrastructure/
COPY backend/Ecommerce.Api/Ecommerce.Api.csproj backend/Ecommerce.Api/
RUN dotnet restore backend/Ecommerce.Api/Ecommerce.Api.csproj
COPY backend/ backend/
```

```
RUN dotnet publish Ecommerce.Api.csproj -c Release -o /app/publish
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
```

What's happening: Copying only .csproj files before restore exploits Docker layer caching: restores rerun only when dependencies change, not on every code edit. Publish -c Release trims debug baggage. The final image sets ASPNETCORE_URLS=http://+:8080 (matching the Nginx upstream and healthcheck) and Production environment. curl exists solely for the /health/db probe.

Why it matters: Slow-but-cached builds and a small runtime image: restore caching is what keeps 'compose build' incremental.

Byte 122: Nginx image and proxy behavior

Source: nginx/Dockerfile; nginx/nginx.conf

Focus: Node webbuild stage and location blocks

Builds on: Bytes 3, 120

In plain terms: The Nginx image first builds the Vue SPA with Node, then serves it while proxying API, health, docs, and Swagger paths to the backend with forwarding headers and timeouts.

The code (nginx):

```
location /api/ {
    proxy_pass http://backend_api;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Authorization $http_authorization;
    proxy_connect_timeout 10s;
    proxy_send_timeout 30s;
    proxy_read_timeout 30s;
    proxy_intercept_errors on;
}
```

What's happening: The Dockerfile's webbuild stage (node:22-alpine, npm ci, npm run build) means the deployed SPA always matches the repo source — no prebuilt dist is committed. Proxy headers preserve client IP/protocol (the API's Secure-cookie decision reads X-Forwarded-Proto via request scheme) and explicitly forward Authorization. Timeouts bound hung backends; intercept_errors lets Nginx render its own error pages for bad-gateway cases. /health, /openapi/v1.json, and /swagger proxy exactly (location = exact match); /nginx-health answers locally for the container healthcheck.

Why it matters: Auth реально depends on two headers here: Authorization forwarded or every authenticated call 401s at the proxy, before .NET ever sees it.

Byte 123: Nginx static caching and SPA fallback

Source: nginx/nginx.conf

Focus: Asset regex block and / fallback

Builds on: Bytes 3, 122

In plain terms: Fingerprinted assets cache for a week as immutable; everything else that is not a file falls back to index.html so client-side routes survive refreshes and deep links.

The code (nginx):

```
gzip on;
gzip_types text/css application/javascript application/json image/svg+xml;
client_max_body_size 5m;
location ~* \.(js|css|png|jpg|jpeg|gif|svg|ico|woff2?)$ {
    expires 7d;
    add_header Cache-Control "public, immutable";
    try_files $uri /index.html;
}
location / {
    try_files $uri $uri/ /index.html;
}
```

What's happening: Regex locations are case-insensitive (~*) and match asset extensions; immutable tells browsers a hashed filename never changes content — safe because Vite fingerprints builds. gzip shrinks text payloads. The 5 MB body cap bounds uploads (product images are URLs, so payloads stay small). try_files order matters: exact file, then directory, then SPA fallback — in that order, so real files always win over the fallback.

Why it matters: Stale-asset bugs after deploy are prevented by fingerprinting + immutable; the fallback is what makes /products/5 refreshable.

Byte 124: Standalone frontend image and dev-only SPA config

Source: frontend/Dockerfile; frontend/nginx-spa.conf

Focus: Three-stage SPA build and fallback-only server block

Builds on: Bytes 72, 73, 122

In plain terms: A separate three-stage Dockerfile builds the SPA alone (deps, build, serve) for standalone static hosting; nginx-spa.conf is the proxy-less variant used outside compose.

The code (dockerfile):

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY frontend/package.json frontend/package-lock.json* ./
RUN npm ci || npm install
FROM node:22-alpine AS build
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY frontend/ ./
RUN npm run build
FROM nginx:1.27-alpine AS final
COPY frontend/nginx-spa.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html
```

What's happening: Splitting deps/build stages caches node_modules separately from source (npm ci preferred, npm install fallback when no lockfile). The standalone image has no /api proxy — it is for static

hosts where the API lives elsewhere (CORS then applies, Byte 36). nginx-spa.conf keeps only gzip + fallback + asset caching. The compose Nginx image (Byte 124) is the one actually deployed with proxying.

Why it matters: Two images, two jobs: compose-Nginx (proxied, deployed) vs standalone (static host, API elsewhere) — pick by where the API runs.

Byte 125: .gitignore: secrets and generated output stay out

Source: .gitignore

Focus: Full file

Builds on: Byte 8

In plain terms: The ignore file blocks secrets (.env), build outputs (bin/obj/dist), dependencies (node_modules), test artifacts, and editor/OS files from ever reaching git.

The code (gitignore):

```
# Secrets \u2014 never commit
.env
# .NET
bin/
obj/
TestResults/
# Node / frontend
node_modules/
frontend/dist/
frontend/coverage/
```

What's happening: Each rule maps to a real artifact in this repo: TestResults/ holds the cobertura coverage XML, frontend/coverage the Vitest HTML report, playwright-report/test-results the E2E runs. *.trx and coverage*.xml catch alternate coverage outputs. The .env rule is the secrets enforcement arm of Byte 8 — .env.example documents, .env itself can never be committed.

Why it matters: Accidentally force-adding .env would publish the database password and JWT key; this file is the guardrail.

PUTTING IT TOGETHER — Section F

PUTTING IT TOGETHER — Section F. SQL Server persists on a named volume behind health gates; the API container carries env-only secrets and proves database health; Nginx builds and serves the SPA while proxying platform paths with auth-preserving headers; asset caching plus SPA fallback keep deep links fast and refreshable; .gitignore keeps secrets and artifacts out of history. Section G shows how all of it is verified: isolated integration tests, mocked store specs, mounted component specs, and browser E2E.

Section G — Testing & Quality

Byte 126: TestFactory: isolated API per test class

Source: backend/Ecommerce.Tests/TestFactory.cs

Focus: WebApplicationFactory with per-class SQLite

Builds on: Bytes 32, 37

In plain terms: Each test class boots the real API against its own uniquely-named in-memory SQLite database, seeded with two categories, three products, and an admin — with cookies disabled so refresh behavior is asserted by hand.

The code (csharp):

```
builder.UseEnvironment("Testing");
builder.UseSetting("TestSqlite", $"DataSource={_dbName};Mode=Memory;Cache=Shared");
builder.ConfigureServices(services =>
{
    _conn = new SqliteConnection($"DataSource={_dbName};Mode=Memory;Cache=Shared");
    _conn.Open();
});
protected static readonly WebApplicationFactoryClientOptions NoCookies = new() { HandleCookies = false
};
```

What's happening: The TestSqlite setting triggers Program.cs's SQLite branch (Byte 32) — a single provider, no dual-registration conflict. Unique DB names per factory stop parallel xUnit classes from sharing rows (a real flake that was fixed). The open shared-cache connection keeps the in-memory DB alive across connections. EnsureReadyAsync uses EnsureCreated (not migrations — the SQL Server migration cannot run on SQLite) with AnyAsync guards for idempotency. Helpers RegisterLoginAsync/AdminClientAsync/UseCart build authed and guest clients; NoCookies forces manual refresh-cookie replay so rotation tests are meaningful.

Why it matters: This harness is why 44 integration tests run in seconds with zero SQL Server — and why refresh-reuse assertions can be trusted.

Byte 127: AuthTests: register to rotation in 8 tests

Source: backend/Ecommerce.Tests/AuthTests.cs

Focus: Registration, login, refresh, logout tests

Builds on: Bytes 45-51, 126

In plain terms: Eight tests walk the auth lifecycle: happy-path register/login/me, duplicate 409, bad-DTO 400, wrong-password 401, rotation with old-token rejection, bogus-token 401, and logout revocation.

The code (csharp):

```
var req1 = new HttpRequestMessage(HttpMethod.Post, "/api/auth/refresh");
req1.Headers.Add("Cookie", $"refresh_token={raw}");
var r1 = await Client.SendAsync(req1);
r1.StatusCode.Should().Be(HttpStatusCode.OK);
// Reuse of rotated token must fail
var req2 = new HttpRequestMessage(HttpMethod.Post, "/api/auth/refresh");
req2.Headers.Add("Cookie", $"refresh_token={raw}");
var r2 = await Client.SendAsync(req2);
r2.StatusCode.Should().Be(HttpStatusCode.Unauthorized);
```

What's happening: Refresh cookies are scraped from Set-Cookie and replayed as Cookie headers manually (NoCookies, Byte 128). The rotation test is the security heart: first refresh 200s, replay of the same token 401s. Logout test proves revocation end to end (refresh after logout 401s). FluentAssertions (Should().Be) reads like prose. CreateAuthed builds Bearer clients from access tokens without touching cookies.

Why it matters: These tests are the executable proof of the rotation guarantee from Byte 47 — if rotation regresses, this suite goes red first.

Byte 128: CatalogTests: public reads and platform proofs

Source: backend/Ecommerce.Tests/CatalogTests.cs

Focus: Category, search, sort, paging, health, OpenAPI tests

Builds on: Bytes 52-55, 126

In plain terms: Eight tests assert categories are public, pagination metadata is coherent, search filters by name, category scoping holds, price sort orders correctly, detail 404s on unknown ids — plus health and OpenAPI endpoints answer.

The code (csharp):

```
var r = await Client.GetAsync("/api/products?sort=price_asc&pageSize=10");
var page = await r.Content.ReadFromJsonAsync<PagedResultDto<ProductCardDto>>();
page!.Items.Select(p => p.Price).Should().BeInAscendingOrder();
```

What's happening: Search test ('phone') asserts every returned item matches in name OR description — mirroring the OR in the service query. Category test scopes by the first live category id (data-driven, not hardcoded). Detail test fetches a real id from pageSize=1 then asserts 999999 404s. Health/OpenAPI tests guard the platform contract Nginx and docs depend on. All anonymous — no auth setup needed.

Why it matters: Storefront display correctness (sort honesty, search precision, paging math) is pinned here, not in the browser.

Byte 129: CartOrderTests: baskets to money in 10 tests

Source: backend/Ecommerce.Tests/CartOrderTests.cs

Focus: Guest cart, merge, checkout, ownership tests

Builds on: Bytes 56-65, 126

In plain terms: Ten tests cover guest add/update/remove, quantity-400, out-of-stock 409, login merge, full checkout (totals math, idempotent retry, stock decrement, cart clear, history+detail), empty-cart 400, bad-method 400, auth 401, and cross-user 403.

The code (csharp):

```
var placed = await authed.PostAsJsonAsync("/api/orders", checkout);
placed.StatusCode.Should().Be(HttpStatusCode.Created);
placed.Headers.Location.Should().NotBeNull();
var order = await placed.Content.ReadFromJsonAsync<OrderDto>();
order!.Total.Should().Be(order.SubTotal + order.ShippingCost + order.TaxAmount);
```

```
// Idempotent retry returns same order
var retry = await authed.PostAsJsonAsync("/api/orders", checkout);
(await retry.Content.ReadFromJsonAsync<OrderDto>())!.Id.Should().Be(order.Id);
```

What's happening: FirstInStockProductIdAsync picks live data (tests never hardcode ids that seeding could change). Guest clients carry X-Cart-Id via UseCart (Byte 128). The checkout test re-reads the product to prove stock dropped by exactly 1 and re-reads the cart to prove it emptied. Ownership test builds an order as user A and asserts user B gets 403. 201 + Location asserted on placement (REST contract).

Why it matters: This is the money-path regression net: oversell, double-charge, cart-loss, and data-leak bugs all have a test with their name on it.

Byte 130: AccountAdminTests: addresses, roles, guarded deletes

Source: backend/Ecommerce.Tests/AccountAdminTests.cs

Focus: Profile, address, admin authorization, CRUD guards, status flow

Builds on: Bytes 66-71, 126

In plain terms: Eight groups assert profile edits, single-default address handling with per-customer isolation, admin 401/403 gating, dashboard numbers, product CRUD, history-blocked deletes, category-blocked deletes, and forward/backward status transitions.

The code (csharp):

```
(await Client.GetAsync("/api/admin/dashboard")).StatusCode.Should().Be(HttpStatusCode.Unauthorized);
var user = await RegisterLoginAsync();
(await user.GetAsync("/api/admin/dashboard")).StatusCode.Should().Be(HttpStatusCode.Forbidden);
var admin = await AdminClientAsync();
var dash = await admin.GetAsync("/api/admin/dashboard");
dash.StatusCode.Should().Be(HttpStatusCode.OK);
```

What's happening: The triple (anonymous 401 / customer 403 / admin 200) pins the policy wiring from Byte 71. Address tests create two defaults and assert exactly one survives, then delete the default and assert promotion. Delete-block tests place a real order first, then assert 409 on product and category deletes. Status test asserts the forward jump (Placed->Delivered) succeeds while the backward move 409s — documenting the jump-allowed semantics.

Why it matters: Authorization and delete-guard regressions are silent data-safety failures; these tests make them loud.

Byte 131: PricingTests: 21 pure unit cases, zero I/O

Source: backend/Ecommerce.Tests/PricingTests.cs

Focus: Theory data for shipping, tax, discount, transitions

Builds on: Bytes 22, 23

In plain terms: Twenty-one [Theory] cases pin every shipping tier boundary, the 8% tax, discount math, and eight status transitions — no database, no server, milliseconds to run.

The code (csharp):

```
[Theory]
[InlineData(0, "standard", 0)]
[InlineData(10, "standard", 5.99)]
[InlineData(74.99, "standard", 5.99)]
[InlineData(75, "standard", 0)]
[InlineData(100, "express", 12.99)]
[InlineData(150, "express", 0)]
public void ShippingFor_Tiers(decimal subtotal, string method, decimal expected) =>
    PricingRules.ShippingFor(subtotal, method).Should().Be(expected);
```

What's happening: Boundary values (74.99 vs 75, 150, 200) test the exact free-shipping edges where off-by-one bugs live. Discount null-cases (no compare, equal prices) pin the 'no badge' behavior. Transition cases include the surprising-but-legal Placed->Delivered jump. xUnit Theory + InlineData keeps each case a one-liner; FluentAssertions gives readable failures.

Why it matters: Cheapest tests in the repo guard the most money-sensitive code — change a tier and this file tells you instantly.

Byte 132: Frontend store specs: Pinia logic without a server

Source: frontend/src/stores/__tests__/auth.spec.ts; frontend/src/stores/__tests__/cart.spec.ts; frontend/src/stores/__tests__/catalog.spec.ts; frontend/src/stores/__tests__/orders.spec.ts

Focus: Mocked-apiClient store tests

Builds on: Bytes 89-92

In plain terms: Four spec files mock the api client and assert each store sends the right calls and syncs the right state — login stores tokens, cart CRUD hits the right URLs, catalog forwards filters, orders attach idempotency keys.

The code (typescript):

```
vi.mock('@/api/client', () => ({
  apiClient: { get: vi.fn(), post: vi.fn(), put: vi.fn(), delete: vi.fn() },
  setAccessToken: vi.fn(),
}))
it('placeOrder attaches idempotencyKey', async () => {
  const result = await store.placeOrder({ shippingAddress: {...}, shippingMethod: 'standard',
paymentMethod: 'cod' })
  expect(post).toHaveBeenCalledWith('/orders', expect.objectContaining({ idempotencyKey:
expect.any(String) }))
})
```

What's happening: vi.mock hoists above imports so stores receive the mock automatically; setActivePinia(createPinia()) gives each test a fresh store. Assertions check both wire shape (URL + payload) and state sync (itemCount, page, current). Catalog spec asserts empty filters are omitted from params. Auth spec asserts logout clears state even when the API rejects — the fail-closed contract. 82.3% store statement coverage (cart + catalog 100%).

Why it matters: Store logic (the frontend's business layer) is fully pinned without spinning a browser — fast feedback on every edit.

Byte 133: Component, guard, and validator specs

Source: frontend/src/components/__tests__/ProductCard.spec.ts; frontend/src/components/__tests__/Pagination.spec.ts; frontend/src/components/__tests__/LoginForm.spec.ts; frontend/src/components/__tests__/AdminOrderTable.spec.ts; frontend/src/router/__tests__/guards.spec.ts; frontend/src/utils/__tests__/validators.spec.ts

Focus: Mount-based specs and pure-function specs

Builds on: Bytes 78, 88, 96, 101, 106

In plain terms: Six spec files mount components with Vue Test Utils (rendering, badges, emits, validation gating) and call pure functions directly (route guards, email/length validators, INR formatting).

The code (typescript):

```
const wrapper = mount(ProductCard, { props: { product }, global: { plugins: [router] } })
expect(wrapper.text()).toContain('Test Phone')
// ...
const res = checkAccess(
  { meta: { requiresAuth: true }, fullPath: '/checkout' } as never,
  { isAuthenticated: false, isAdmin: false })
expect(res.redirect).toBe('/login')
```

What's happening: ProductCard mounts inside a stub router (RouterLink needs routing context). Pagination asserts emitted page numbers, not DOM следствие. LoginForm asserts submit is suppressed for bad input and payload shape for good input. AdminOrderTable asserts the status select emits {id, status}. Guard specs cover all five branches of checkAccess because it is pure. Validators specs pin email/required/length plus the ₹ symbol in formatted output.

Why it matters: Presentation and routing rules are executable documentation: change badge logic or guard order and a spec names the regression.

Byte 134: E2E smoke: six tests against the real stack

Source: frontend/e2e/smoke.spec.ts; frontend/playwright.config.ts; frontend/playwright.prod.config.ts

Focus: Playwright flows and both configs

Builds on: Bytes 94-117

In plain terms: Six Chromium tests drive the deployed app like a shopper: browse/search/details, add-to-cart, register-to-checkout-to-history, seeded login, admin create-product, and non-admin block — with data-testid selectors throughout.

The code (typescript):

```
await page.locator('#pl-search').fill('phone')
await page.locator('#pl-search').press('Enter')
await expect(page).toHaveURL(/search=phone/)
const cards = page.locator('[data-testid="product-card"]')
await expect(cards.first()).toBeVisible({ timeout: 15000 })
```

```
// Checkout requires cart items: add one product first.
await page.locator('#ship-fullName').fill('E2E User')
await page.locator('[data-testid="checkout-form"] button[type="submit"]').click()
await expect(page.locator('[data-testid="order-confirmation"]')).toBeVisible({ timeout: 20000 })
```

What's happening: Dev config boots Vite automatically (webServer) against :5173 + proxied :5000; prod config reuses the same suite with webServer disabled against the Docker stack at :8080 — one suite, two targets. Unique e2e-${Date.now()} emails avoid collisions. Generous timeouts (15-20s) absorb cold container starts. The suite caught two real bugs: the auth-restore race (fixed in main.ts) and null billing (fixed in CheckoutView). Six tests cover eleven required scenarios by chaining flows.

Why it matters: Unit tests verify parts; only this suite verifies the running system — the reflection notes' central lesson.

Byte 135: TESTING.md: scores, coverage, and incident log

Source: docs/TESTING.md

Focus: Counts, coverage table, bug lists

Builds on: Bytes 126-134

In plain terms: The testing doc records 52 backend + 38 frontend + 6 E2E tests all passing, measured coverage versus targets, every real bug the suites caught, and one production incident with its root cause.

The code (markdown):

```
Measured line coverage: **Ecommerce.Core 92 %** (target \u2265 70 \u2713),
**Ecommerce.Api 70.6 %** (services + endpoints), **Ecommerce.Infrastructure 40.7 %**
(mostly generated migration code). Business rules + services exceed the target.
Real bugs found by these tests and fixed: JWT placeholder key too short for HS256
(test env), dual EF provider registration in the factory ... and a `CfgOrEnv` fallback bug.
```

What's happening: Coverage is reported where it matters (Core 92%, stores 82.3%) rather than chased globally — views are E2E-covered, migrations excluded as generated. The doc lists six backend test bugs and two E2E bugs with fixes, plus the CfgOrEnv signing/validation divergence incident (config->env->fallback ordering). 'Not run / limitations' honestly scopes what was skipped (dev-config Playwright, load/pen testing, CI).

Why it matters: This file is the quality certificate and the debugging diary in one — read it before doubting a green suite or repeating a fixed mistake.

PUTTING IT TOGETHER — Section G

PUTTING IT TOGETHER — Section G. Per-class SQLite harnesses boot the real API in seconds; auth, catalog, cart/order, account/admin, and pricing suites pin every backend promise including rotation, idempotency, ownership, and delete guards; Vitest specs pin stores, components, guards, and validators; Playwright drives the deployed stack end to end and already caught two production-class bugs; TESTING.md keeps the score. Section H collects the sharp edges these layers leave behind.

Section H — Edge Cases & Integration

Byte 136: MIGRATION.md: legacy-to-modern feature map

Source: docs/MIGRATION.md

Focus: Capability table and gap list

Builds on: Bytes 1, 12-23

In plain terms: Fourteen legacy capabilities mapped to their new homes (Vue page, Pinia store, API endpoint, service) plus five deliberate architecture changes and five documented gaps.

The code (markdown):

```
| 7 | Shipping tiers: standard 5.99 (free \u2265 75), express 12.99 (free \u2265 150), priority 24.99
(free \u2265 200); tax 8 % | `PricingRules` (`Mapping.cs`) | Copied verbatim into
`Ecommerce.Core.PricingRules` |
| 10 | Identity registration/login ... Admin role via `AspNetUserRoles` | `AccountController` ... |
Self-contained `Customer` ... JWT + refresh tokens replace cookie auth |
```

What's happening: Each row names the legacy location (service/table) and the preserved rule — the audit trail that behaviors survived the port. Deliberate changes (session->persisted cart, cookie->JWT, EF6->EF Core, Razor->SPA) explain why flows look different while rules match. Known gaps are explicit: no variants/galleries, flat categories, no guest checkout, trimmed dashboard series, INR-only prices — each with a re-extension sketch.

Why it matters: Scope arguments end here: 'is X supposed to work?' is answered by row lookup, not opinion.

Byte 137: API.md, REFLECTION.md, DEMO_SCRIPT.md: contract, lessons, tour

Source: docs/API.md; docs/REFLECTION.md; docs/DEMO_SCRIPT.md

Focus: Roles of the three remaining docs

Builds on: Bytes 50-71

In plain terms: API.md is the endpoint contract (routes, bodies, statuses); REFLECTION.md records the state-relocation lesson and both production bugs; DEMO_SCRIPT.md is the timed walkthrough with fallback one-liners.

The code (markdown):

```
| POST | `/api/auth/refresh` | `{refreshToken?}` (or HttpOnly cookie) \u2192 200 `AuthResponse`
(rotates) |
- `GET /api/orders/{id}` \u2192 `OrderDto{...}` (403 for other customers' orders)
## 4. Register/login + merge (1.5 min)
- Register a new user (unique e-mail) \u2192 auto-login, guest items retained (merged).
```

What's happening: API.md's tables pin every route, DTO shape, and status code convention (201+Location, 204 deletes, 400/401/403/404/409/429) — the frontend-backend handshake in one page. REFLECTION.md distills the port into 'state relocation' (session cart, cookie auth, server pages each moved somewhere

honest) and the verify-the-running-system rule. DEMO_SCRIPT.md sequences an 8-part demo with seeded logins and per-minute beats plus break-glass curl/test fallbacks.

Why it matters: Contract, wisdom, and demo in three files: integrate against the first, onboard with the second, present with the third.

Byte 138: Edge: stolen refresh tokens and dead sessions

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs; frontend/src/api/client.ts

Focus: Rotation reuse + interceptor fallback

Builds on: Bytes 47, 83, 127

In plain terms: A reused refresh token always 401s (theft signal, by design), and a dead session clears auth and hard-routes to login — with one known cost: two tabs refreshing at once can log one of them out.

The code (csharp):

```
if (stored == null || !stored.IsActive || stored.Customer == null)
    throw new DomainException("Invalid or expired refresh token.", 401);
```

What's happening: Reuse detection has no grace window: whichever request arrives second loses, legitimate or not. The frontend's single silent retry (Byte 83) usually masks expiry, but concurrent 401s in two tabs both attempt refresh with the same cookie — the loser gets logged out despite doing nothing wrong. No server-side reuse alarm exists (no logging/alert on reuse), a documented future-work item. Expired-but-unrevoked rows accumulate until refresh-token cleanup (none implemented — table grows).

Why it matters: If users report 'random logouts,' check concurrent-refresh races before suspecting token theft — and know the table needs a future purge job.

Byte 139: Edge: stock races from cart to checkout

Source: backend/Ecommerce.Api/Services/CatalogCartAuth.cs; backend/Ecommerce.Api/Services/OrderAccountAdmin.cs

Focus: Add/update caps vs transactional re-check

Builds on: Bytes 57, 58, 62, 63, 129

In plain terms: Cart writes check stock at write time, but stock moves — so checkout re-checks inside a transaction and 409s the whole order on shortage rather than overselling or short-shipping.

The code (csharp):

```
if (newQty > product.Stock) throw new DomainException($"Only {product.Stock} unit(s) available.", 409);
// ... at checkout, inside the transaction:
if (ci.Quantity > p.Stock) throw new DomainException($"Only {p.Stock} unit(s) of '{p.Name}'
available.", 409);
```

What's happening: Between cart-add and checkout, other shoppers buy and admins edit stock — the cart's StockAvailable display can go stale (the row component only refreshes on fetch). True concurrent oversell (two checkouts, one unit left) is rejected for the loser by the in-transaction check; without row-level locking

there is a narrow race where both read before either writes — acceptable at this scale, documented as 'rejected beyond available stock'. Zero-stock adds 409 at the cart so dead items never reach checkout. Merge caps at min(stock,99) for the same reason.

Why it matters: Every 'only N available' message the shopper sees originates in one of these two checks — same wording family, two enforcement points.

Byte 140: Edge: ownership, roles, and delete guards

Source: backend/Ecommerce.Api/Services/OrderAccountAdmin.cs; backend/Ecommerce.Api/Endpoints/Endpoints.cs

Focus: GetDetail 403, Admin 403, delete 409s

Builds on: Bytes 64, 67, 69-71, 130

In plain terms: Cross-customer order reads 403, non-admins 403 on every /api/admin route, and deletes of referenced products/categories 409 with 'deactivate instead' guidance — each enforced server-side, never trusted to the UI.

The code (csharp):

```
if (!isAdmin && o.CustomerId != customerId) throw new DomainException("Forbidden.", 403);
// AdminService:
if (await db.OrderItems.AnyAsync(i => i.ProductId == id, ct))
    throw new DomainException("Product has historical orders and cannot be deleted. Deactivate it
instead.", 409);
```

What's happening: The 403 message is deliberately bare ('Forbidden.') to avoid confirming order existence to probers. Admin 403 vs 401 distinguishes 'logged in, wrong role' from 'not logged in' — the frontend guards mirror this (home vs login redirects). Delete guards check carts too (a product sitting in any live cart blocks deletion). Category delete checks products; address delete of a foreign id 404s (indistinguishable from missing — no leak). The frontend hides admin links, but hiding is not enforcement — direct /api/admin calls still 403.

Why it matters: IDOR (insecure direct object reference) and privilege-escalation classes are closed by these checks plus their tests — UI hiding alone would leave them open.

Byte 141: Edge: startup ordering and the two real incidents

Source: backend/Ecommerce.Api/Program.cs; frontend/src/main.ts; docs/TESTING.md

Focus: Health gates, router race, CfgOrEnv divergence

Builds on: Bytes 5, 37, 76, 135

In plain terms: Three boot-time failure modes were found and fixed: database-not-ready (health gates), router-before-refresh (auth-first boot), and signing/validation key divergence (CfgOrEnv ordering) — each with a distinct symptom.

The code (markdown):

```
After `docker compose up`, authenticated endpoints returned 401 for freshly
issued tokens. Cause: `appsettings.json` carries a literal `"${JWT_KEY}"`
placeholder; the `CfgOrEnv` helper returned the dev fallback whenever the config
value was non-null \u2014 never consulting the environment \u2014 while `TokenService`
resolved the real env key. Signing (64 chars) and validation (50-char fallback)
diverged in the same process.
```

What's happening: Symptom map: empty store + api restarting = db not ready (wait 40-120s, check 'docker compose logs api'); protected deep-links bounce to login on refresh = router race (fixed by Byte 76 ordering — regresses if anyone reorders main.ts); ALL authenticated calls 401 with fresh tokens = key divergence (diff Program.cs vs TokenService resolution). Port clash on 8080 = set HOST_PORT. Windows long-path restore failures = shorten repo path. Legacy stack coexistence holds via isolated names/networks/volumes.

Why it matters: This byte is the on-call runbook: three incidents, three signatures, three fixes — diagnose by symptom, not by guessing.

PUTTING IT TOGETHER — Section H

PUTTING IT TOGETHER — Section H. Rotation makes token reuse fail closed; stock is checked at cart time and re-checked inside the checkout transaction; ownership, roles, and history guards hold server-side no matter what the UI hides; and the three boot incidents (health gates, router race, key divergence) each have a symptom and a fix. The legacy map and doc roles close the loop on where everything came from and where to look next.

Glossary

SPA (Single-Page Application): One HTML shell whose views render in the browser; navigation never reloads the page.

Minimal API: .NET style defining HTTP endpoints as functions without controllers.

EF Core: Entity Framework Core: object-relational mapper turning C# classes into SQL tables and queries.

Migration: Versioned code describing schema changes; applied at startup to converge the database.

DTO: Data Transfer Object: shaped payload crossing the API boundary, never a database entity.

Pinia: Vue state-management library; stores hold shared state like auth, cart, catalog.

JWT: JSON Web Token: signed, short-lived credential carried in the Authorization header.

Refresh rotation: Each refresh token works once: use retires it and mints a replacement; reuse fails.

HttpOnly cookie: Cookie invisible to JavaScript, mitigating theft via cross-site scripting.

CORS: Cross-Origin Resource Sharing: server allow-list deciding which browser origins may call the API.

Idempotency key: Client token making retried operations return the original result instead of duplicating.

Snapshot: Frozen copy of data (address, price) stored on the order so later edits cannot rewrite history.

ProblemDetails: RFC 9457 JSON error envelope (type/title/status/detail) returned by the API.

FluentValidation: .NET library expressing request rules as code; failures become 400 validation problems.

Rate limiting: Per-IP request throttling (20/min on auth) against brute force and abuse.

OpenAPI: Machine-readable API description served at /openapi/v1.json; drives Swagger UI.

Healthcheck: Container probe (db query, /health/db, Nginx ping) gating startup order.

Reverse proxy: Nginx forwarding /api, /health, /openapi paths to the backend container.

History-mode fallback: Serving index.html for unknown paths so client-side routes survive refresh.

Filtered index: Unique database index ignoring NULLs, enabling optional-but-unique owners (guest/user carts).

Slug: URL-friendly identifier derived from a name (Banarasi Silk Saree becomes banarasi-silk-saree).

Seed data: Demo categories, products, and logins inserted idempotently at startup.

WebApplicationFactory: xUnit harness booting the real API in-process against an isolated test database.

DomainException: Business-rule failure carrying its HTTP status; endpoints map it to ProblemDetails.

Claims: Name/value facts inside a JWT (user id, email, role) that endpoints read for identity.

Bearer token: Access token sent as 'Authorization: Bearer <token>' on protected calls.

E2E (end-to-end): Playwright tests driving a real browser against the deployed stack.

Vitest: Frontend unit/component test runner used with Vue Test Utils and jsdom.

PUTTING IT TOGETHER — the whole system

A shopper arrives at Nginx, which serves the Vue shell and proxies platform paths to the Minimal API. The shell boots auth-first, guards routes purely, and tags every call with memory token plus guest cart id. Catalog reads are public and deterministic; carts resolve guest-or-user and guard stock; checkout deduplicates and commits atomically with frozen snapshots; accounts keep one default address; admins work behind a role policy with history-safe deletes and a forward-only order machine. SQL Server persists it all on a named volume; health-gated Compose starts db, then api (migrate + seed), then Nginx. Fifty-two backend tests, thirty-eight frontend tests, and six browser E2E tests pin the promises, and the edge-case section records where the guarantees bend: concurrent refresh races, stale stock displays, UI-hidden-but-server-enforced authorization, and the three boot incidents with their signatures. That is the full picture reassembled: every byte a piece, every chapter a cluster, every flow traceable from click to row.